

TRÌNH BIÊN DỊCH

Chương 1. Phân tích từ vựng

Phạm Nguyễn Khang
BM. Khoa học máy tính

pnkhang@cit.ctu.edu.vn

CẦN THƠ, 12 - 2012

```
#include ...  
int main() {  
    ...  
}
```



Phân tích từ vựng

Phân tích cú pháp

Phân tích ngữ nghĩa

Sinh mã trung gian

Tối ưu mã trung gian

Sinh mã

Tối ưu



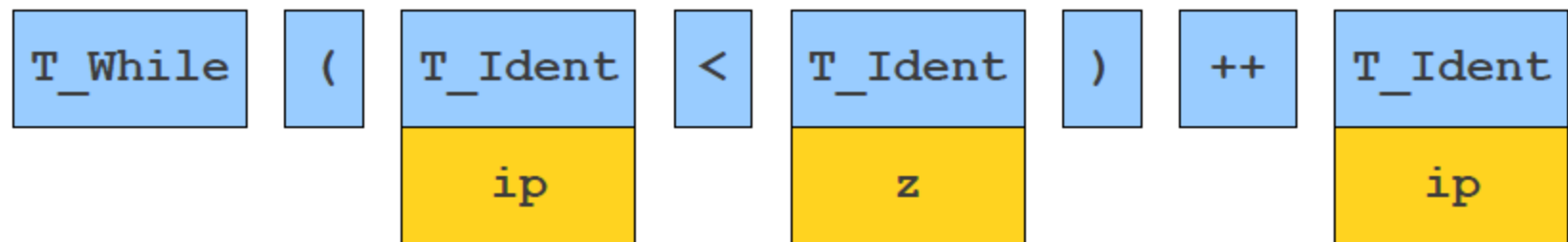
```
0001001000  
1011011100  
1100101111  
1000011111
```

```
while (ip < z)  
    ++ip;
```

w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;
---	---	---	---	---	--	---	---	---	--	---	--	---	---	----	----	---	---	---	---	---

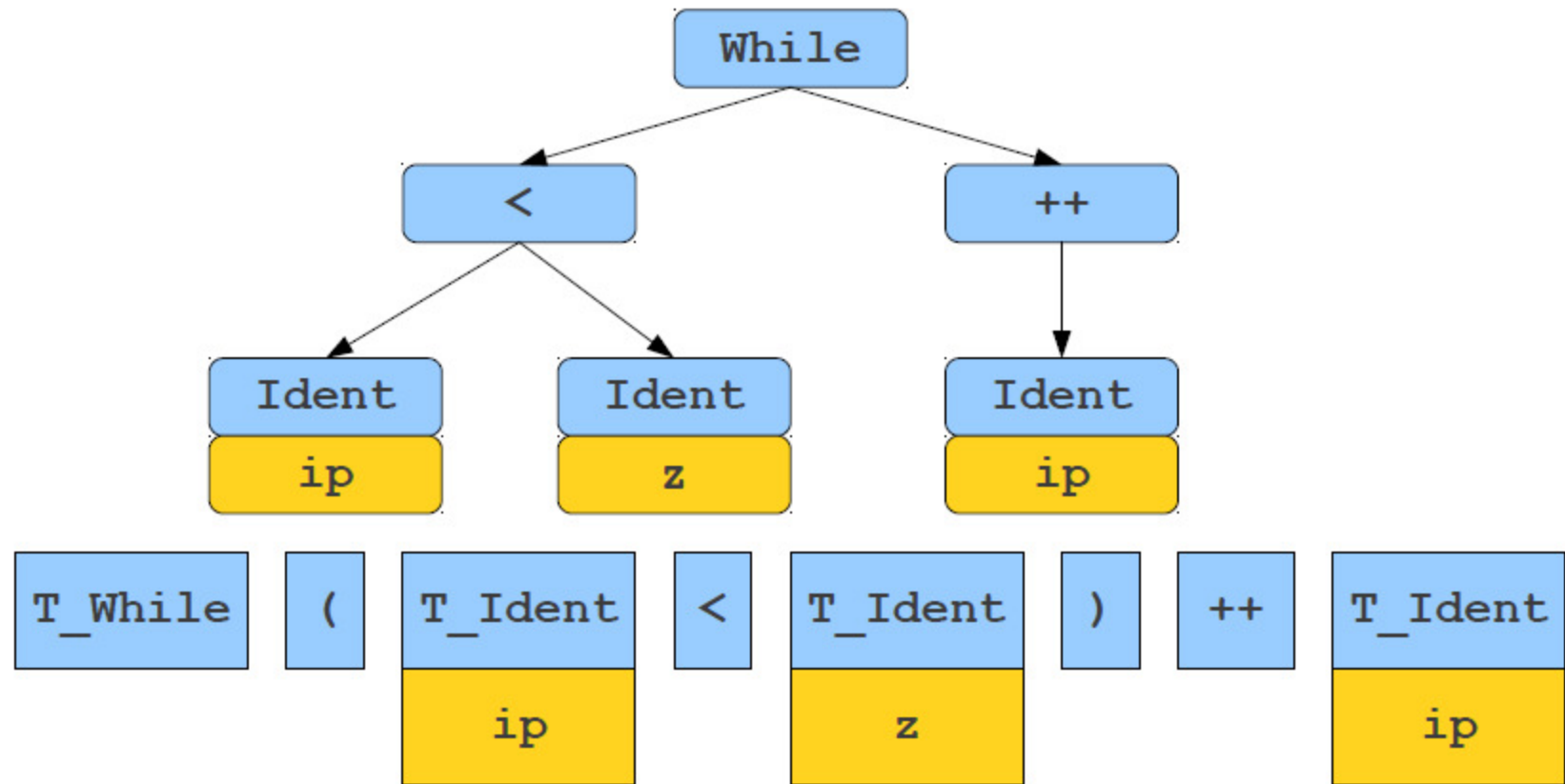
```
while (ip < z)
    ++ip;
```

Token



w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;
---	---	---	---	---	--	---	---	---	--	---	--	---	---	----	----	---	---	---	---	---

```
while (ip < z)
    ++ip;
```



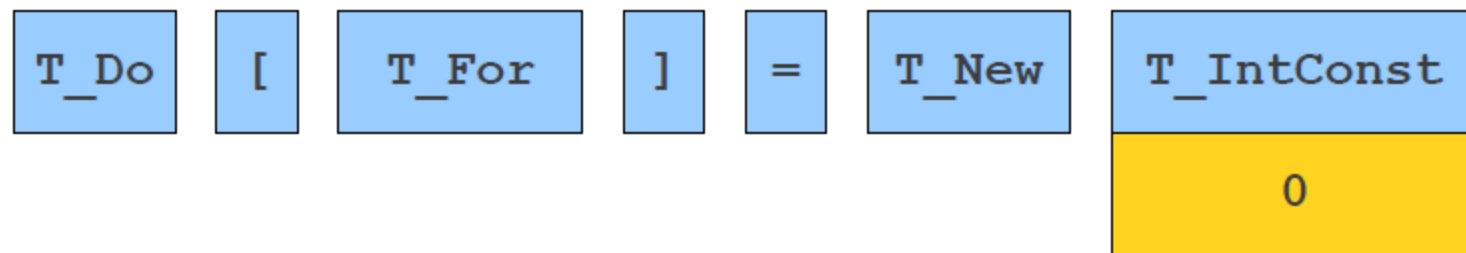
w	h	i	l	e		(	i	p		<		z	)	\n	\t	+	+	i	p	;
---	---	---	---	---	--	---	---	---	--	---	--	---	---	----	----	---	---	---	---	---

```
while (ip < z)
    ++ip;
```

```
do[for] = new 0;
```

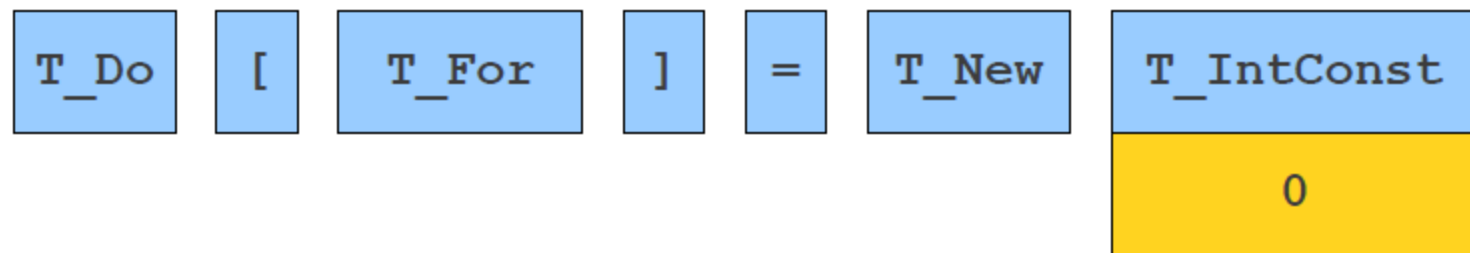
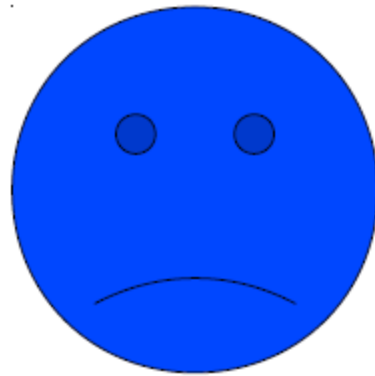
d	o	[	f	o	r	]		=		n	e	w		0	;
---	---	---	---	---	---	---	--	---	--	---	---	---	--	---	---

do[for] = new 0;



d o [f o r] = n e w 0 ;

`do[for] = new 0;`



d	o	[	f	o	r	]		=		n	e	w		0	;
---	---	---	---	---	---	---	--	---	--	---	---	---	--	---	---

do[for] = new 0;

Duyệt qua tập tin nguồn (scanning)

w	h	i	l	e		(	1	3	7		<		i	)	\n	\t	+	+	i	;
---	---	---	---	---	--	---	---	---	---	--	---	--	---	---	----	----	---	---	---	---

Duyệt qua tập tin nguồn (scanning)

w	h	i	l	e		(	1	3	7		<		i	)	\n	\t	+	+	i	;
---	---	---	---	---	--	---	---	---	---	--	---	--	---	---	----	----	---	---	---	---

Một phần của chương trình nguồn tạo nên các token gọi là **lexeme** (từ vị).

T_While

Đây được gọi là 1 **token**. Có thể xem đây là một kiểu liệt kê biểu diễn các thành phần logic của một chương trình.

Duyệt qua tập tin nguồn (scanning)

w	h	i	l	e		(	1	3	7		<		i	)	\n	\t	+	+	i	;
---	---	---	---	---	--	---	---	---	---	--	---	--	---	---	----	----	---	---	---	---

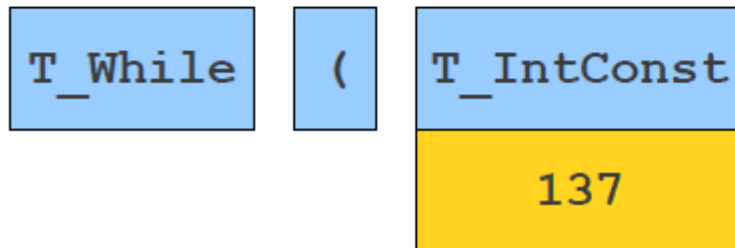
Đôi khi 1 **lexeme** cũng có thể được bỏ qua thay vì lưu trữ để sử dụng về sau.

Trong trường hợp này ta bỏ qua các khoảng trắng vì nó không mang ý nghĩa gì hết đối với chương trình.

T_While

Duyệt qua tập tin nguồn (scanning)

w	h	i	l	e		(	1	3	7		<		i	)	\n	\t	+	+	i	;
---	---	---	---	---	--	---	---	---	---	--	---	--	---	---	----	----	---	---	---	---



Một số token có thể có các **thuộc tính (attributes)** lưu trữ các thông tin bổ sung cho token đó.

Trong ví dụ này, ta lưu số nguyên 137

Mục tiêu của Phân tích từ vựng

- Chuyển từ chương trình nguồn thành một **chuỗi các tokens**.
- Mỗi token biểu diễn một thành phần logic của chương trình nguồn
 - Ví dụ: từ khóa, tên biến, hằng, dấu ngoặc, ...
- Mỗi token tương ứng với một **lexeme (từ vị)** biểu diễn chuỗi ký tự tạo nên token đó.
 - Ví dụ: "137," "int," "for", ...
- Mỗi token có thể có các **thuộc tính (attributes)** để lưu các thông tin bổ sung được trích ra từ lexeme
 - Ví dụ: giá trị số
- **Chuỗi các token** sẽ được dùng để khôi phục/xây dựng lại **cấu trúc của chương trình**.

Lựa chọn tokens

```
for (int k = 0; k < myArray[5]; ++k) {  
    cout << k << endl;  
}
```

- Liệt kê các tokens trong đoạn chương trình nguồn trên

Lựa chọn tokens

```
for (int k = 0; k < myArray[5]; ++k) {  
    cout << k << endl;  
}
```

- for (int = ; <
- [] ++) { <<
- Identifier
- IntegerConstant

Chọn token tốt

- Phụ thuộc rất nhiều vào ngôn ngữ
- Cụ thể, ta có thể chọn:
 - Mỗi từ khóa ứng với 1 token, ví dụ: **for, do, while**
 - Mỗi ký hiệu ngăn cách (dấu câu) ứng với 1 token, ví dụ: **dấu chấm (.) chấm phẩy (;) hai chấm (:)** ...
 - Nhóm các lexemes biểu diễn các định danh (identifiers), hằng số (numeric constants), hằng chuỗi (string constants) vào nhóm tương ứng, mỗi nhóm ứng với 1 token.
- Bỏ qua các thông tin không quan trọng (khoảng trắng, chú thích)

Khó khăn

- Ngôn ngữ FORTRAN: khoảng trắng không quan trọng

DO 5 I = 1, 25

DO5I = 1.25

- Phân hoạch đầu vào **khi nào** ?

Khó khăn

- C++: Khai báo template lồng nhau

```
vector<vector<int>> myVector  
vector < vector < int >> myVector  
(vector < (vector < (int >>  
myVector)) )
```

- Phân hoạch đầu vào **ở đâu (vị trí nào)** ?

Khó khăn

- PL/1: từ khóa có thể dùng như danh biểu (identifiers).

IF THEN **THEN** THEN = ELSE; **ELSE** ELSE = IF

- Gán nhãn cho các lexemes **như thế nào** ?

Những thách thức

- Gán lexemes nào cho token nào ?
- Khi có nhiều cách duyệt, chọn cách nào cho phù hợp ?
- Làm sao giải quyết các vấn đề trên 1 cách hiệu quả (đúng đắn và nhanh chóng) ?

Lexemes và tokens

- Sử dụng tokens để phân loại lexemes
- Một token có thể tương ứng với 1 lexeme duy nhất:
 - Tokens từ khóa như `if`, `while`, `do`.
- Một token có thể tương ứng với nhiều lexemes:
 - Tên biến, số, chuỗi, ...

Tập lexemes

- Ý tưởng: gán một tập lexemes vào một token.
- Ví dụ:
 - Token **"number"** có tập lexemes { **0, 1, 2, ..., 10, 11, 12, ...** } **"[0-9]+"**
 - Token **"string"** được liên kết với tập lexemes { **"", "a", "b", "c", ...** } **"[^\"]*"**
 - Token **"while"** có tập lexemes { **while** }.

Làm thế nào mô tả tập lexemes (có thể có vô hạn phần tử) cho các tokens tương ứng ?

Ngôn ngữ hình thức

- Ngôn ngữ hình thức (formal language) là một tập các chuỗi
- Nhiều ngôn ngữ vô hạn được định nghĩa bằng tập hữu hạn các mô tả:
 - Định nghĩa ngôn ngữ bằng automata
 - Định nghĩa ngôn ngữ bằng văn phạm
 - Định nghĩa ngôn ngữ bằng biểu thức chính quy

Biểu thức chính quy

- Biểu thức chính quy là một họ các mô tả dùng để định nghĩa một số ngôn ngữ (ngôn ngữ chính quy)
- Cung cấp một mô tả ngắn gọn dễ hiểu (humanreadable) cho ngôn ngữ
- Được sử dụng như công cụ cơ bản trong nhiều hệ thống phần mềm, trong đó có công cụ flex (ta sẽ sử dụng công cụ này trong phần thực hành/demo).

Biểu thức chính quy nguyên tố

- Các biểu thức chính quy sử dụng trong bài giảng này được xây dựng từ 2 khối nguyên tố:
 - ϵ là một biểu thức chính quy cho chuỗi rỗng.
 - Ký hiệu a là biểu thức chính quy cho chuỗi "a".

Biểu thức chính quy phức

- Nếu R_1 và R_2 là biểu thức chính quy thì R_1R_2 là **biểu thức chính quy** biểu diễn **phép ghép nối (concatenation)** hai của ngôn ngữ của R_1 và của R_2 .
- Nếu R_1 và R_2 là biểu thức chính quy thì $R_1 \mid R_2$ là **biểu thức chính quy** biểu diễn **phép hợp (union)** hai của ngôn ngữ của R_1 và của R_2 .
- Nếu R là một biểu thức chính quy, R^* là **biểu thức chính quy bao đóng Kleene** của R .
- Nếu R là biểu thức chính quy (R) cũng là **biểu thức chính quy có ý nghĩa giống như R** .

Thứ tự ưu tiên của các phép toán

- Thứ tự ưu tiên của các phép toán trong biểu thức chính quy như sau:

(R)

R^*

R_1R_2

$R_1 \mid R_2$

- Ví dụ **$ab^*c \mid d$** được hiểu là **$((a(b^*)))c \mid d$**

Một số biểu thức chính quy đơn giản

- Cho biểu thức chính quy:

$(0 \mid 1)^*00(0 \mid 1)^*$

- Các chuỗi sau đây chuỗi nào có thể được sinh ra từ từ biểu thức trên

a)11011100101

b)0000

c)1111

d)11111011111011111

e)10110011111

Một số biểu thức chính quy đơn giản

- Biểu thức chính quy:

$(0|1)(0|1)(0|1)(0|1)$

biểu diễn cho các chuỗi nào ?

- Biểu thức trên tương đương với

$(0|1)\{4\}$

- Biểu thức

$1^*(0|\epsilon)1^*$

biểu diễn cho các chuỗi nào ? Cho 4 ví dụ

- Biểu thức trên cũng tương đương với

$1^*0?1^*$

Một số biểu thức chính quy đơn giản

- Giả sử **a** là một ký tự chữ cái hoặc chữ số bất kỳ. Hãy viết một **biểu thức chính quy** mô tả địa chỉ email.
- Viết biểu thức chính quy mô tả các số nguyên chẵn:

$(+|-|\epsilon)(0|1|2|3|4|5|6|7|8|9)^*(0|2|4|6|8)$

Biểu thức trên tương đương với:

$(+|-)?[0123456789]^*[02468]$

apt-get install flex bison

Mẫu (Patterns) trong Flex

- Flex sử dụng **biểu thức chính quy mở rộng** để định nghĩa các mẫu (patterns)
 - x ký tự x
 - . bất kỳ ký tự nào
 - [xyz] ký tự x hoặc y hoặc z
 - [abj-oZ] ký tự a, b, j, k, l, m, n, o hoặc Z
 - [0-9] ký tự 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
 - [^abc] ký tự nào cũng được trừ a, b và c
 - [^A-Z] bất kỳ ký tự nào trừ các ký tự từ A đến Z
 - [^A-Z\n] trừ các ký tự in hoa và xuống dòng

Mẫu (Patterns) trong Flex

- Flex sử dụng **biểu thức chính quy mở rộng** để định nghĩa các mẫu (patterns)
 - r^* 0 hoặc nhiều ký tự r
 - r^+ 1 hoặc nhiều ký tự r
 - $r?$ 0 hoặc 1 ký tự r
 - $r\{2,5\}$ từ 2 đến 5 ký tự r
 - $r\{2,\}$ ít nhất 2 ký tự r
 - $r\{4\}$ 4 ký tự r
 - `"xysd\"ef"` chuỗi ký tự **xysd "ef"**
 - `\x` ($x = 'a', 'b', 'f', 'n', 'r', 't', \text{ or } 'v'$) có ý nghĩa như ký tự `\` trong ngôn ngữ C

Mẫu (Patterns) trong Flex

- Flex sử dụng **biểu thức chính quy mở rộng** để định nghĩa các mẫu (patterns)
 - \0 ký tự có mã ASCII 0
 - \123 ký tự có mã 123 (hệ 8)
 - \x2a ký tự có mã 2a (hệ 16)
 - (r) tương đương với biểu thức r
 - rs phép toán ghép biểu thức r và s
 - r|s phép toán hoặc của biểu thức r và s
 - r/s biểu thức r theo sau bởi s
 - ^r biểu thức r ở đầu hàng
 - r\$ biểu thức r ở cuối hàng

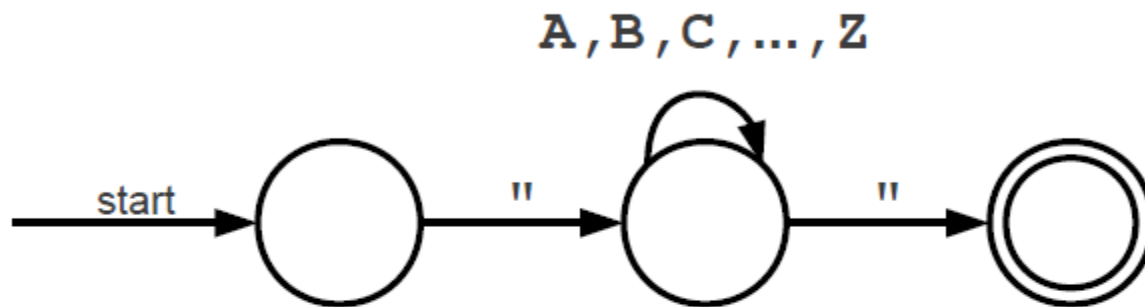
Mẫu (Patterns) trong Flex

- Flex sử dụng **biểu thức chính quy mở rộng** để định nghĩa các mẫu (patterns)
 - <<EOF>> kết thúc file

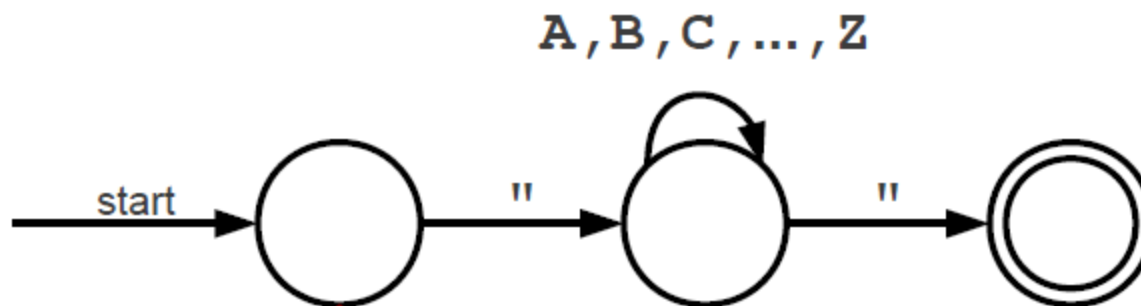
Cài đặt biểu thức chính quy

- Biểu thức chính quy thường được cài đặt bằng **automat hữu hạn (finit automata)**
- Có 2 loại automat hữu hạn
 - Không đơn định (nondeterministic): NFA
 - Đơn định (nondeterministic): DFA

Automat (ví dụ)

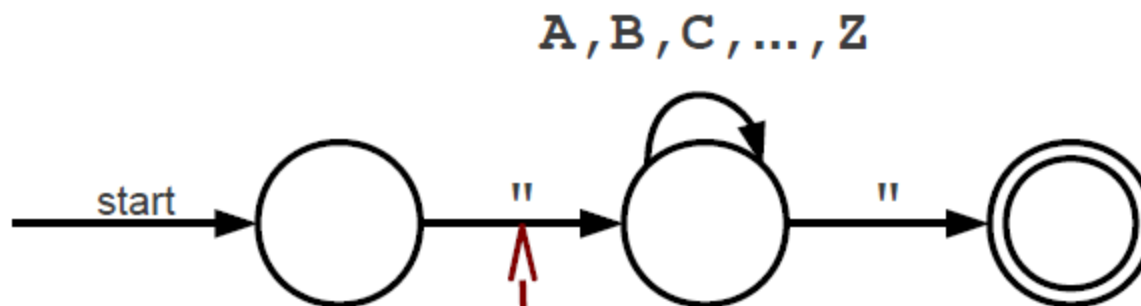


Automat (ví dụ)



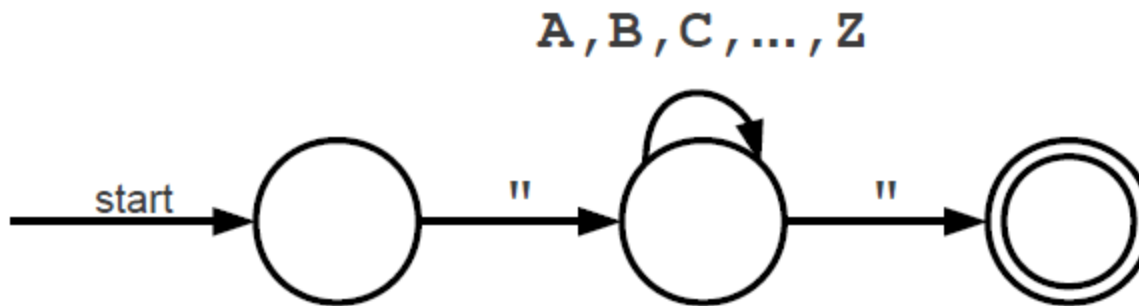
Mỗi vòng tròn tương ứng với một **trạng thái (state)**. Cấu hình của automat được xác định bằng các trạng thái hiện hành.

Automat (ví dụ)



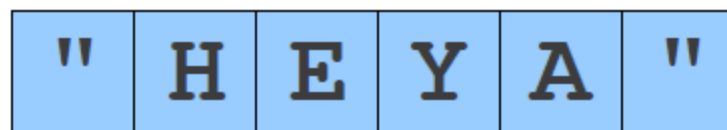
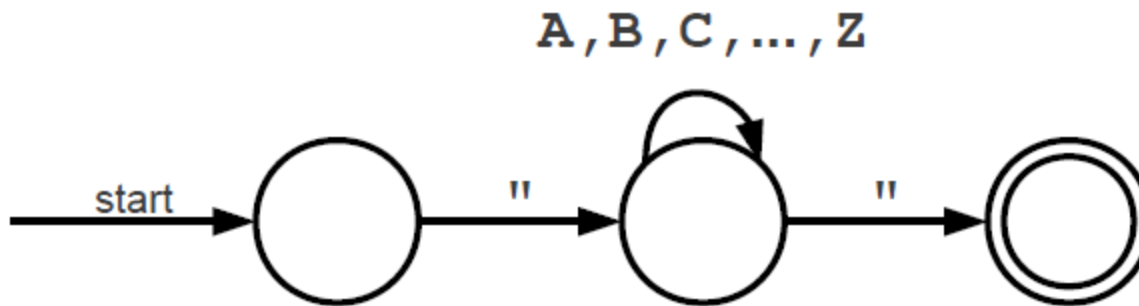
Mỗi mũi tên tương ứng với một **hàm chuyển trạng thái (transition)**. Automat chuyển thái dựa trên trạng thái hiện hành và hàm chuyển trạng thái.

Automat (ví dụ)



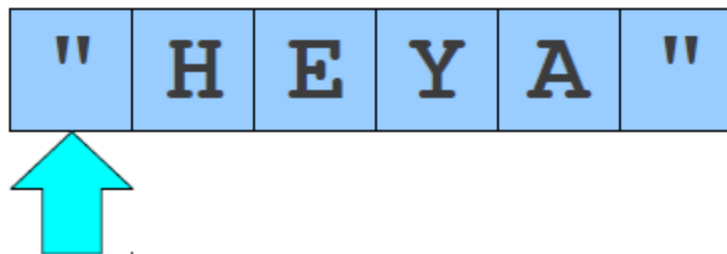
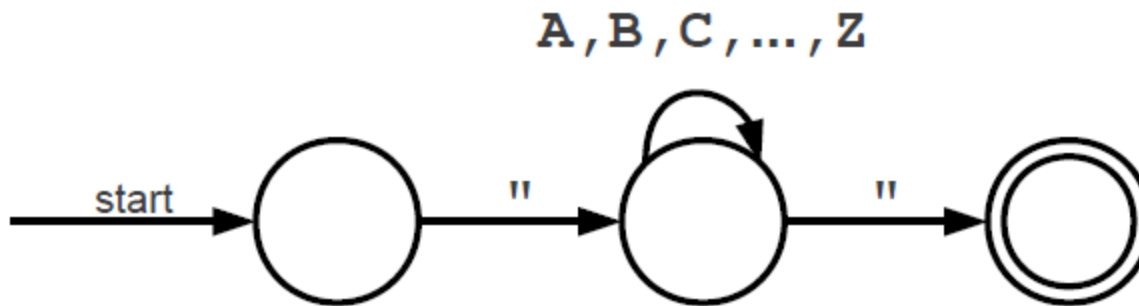
"	H	E	Y	A	"
---	---	---	---	---	---

Automat (ví dụ)

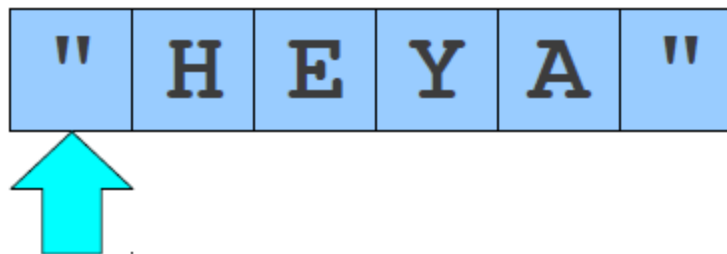
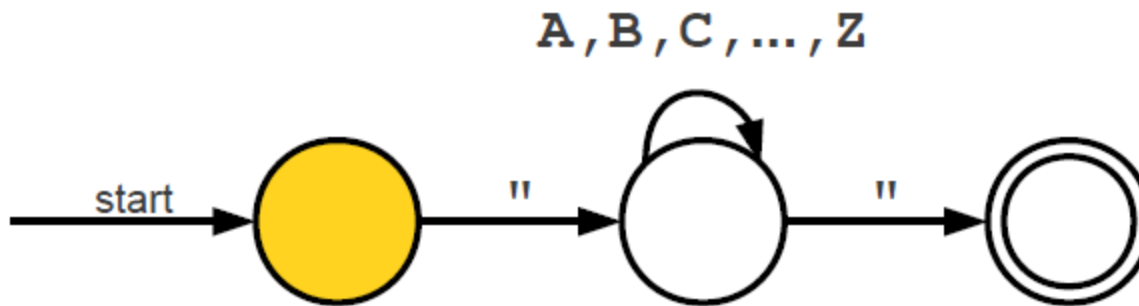


Automat nhận đầu vào là một chuỗi và xác định là **chấp nhận** hay **bác bỏ** chuỗi này.

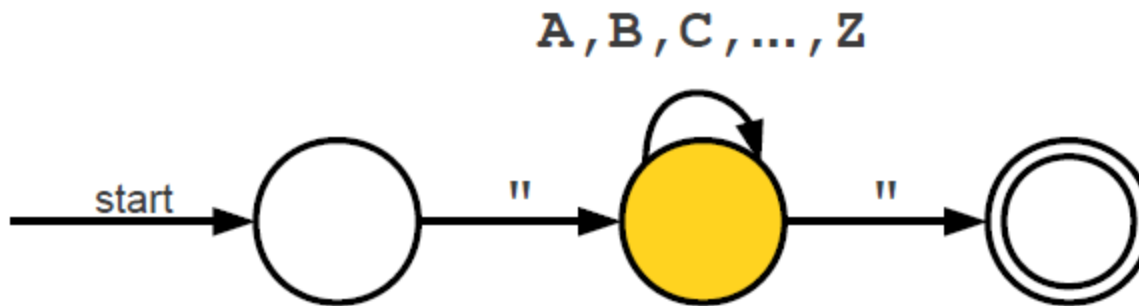
Automat (ví dụ)



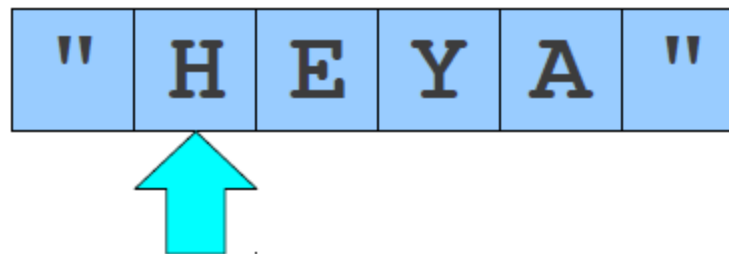
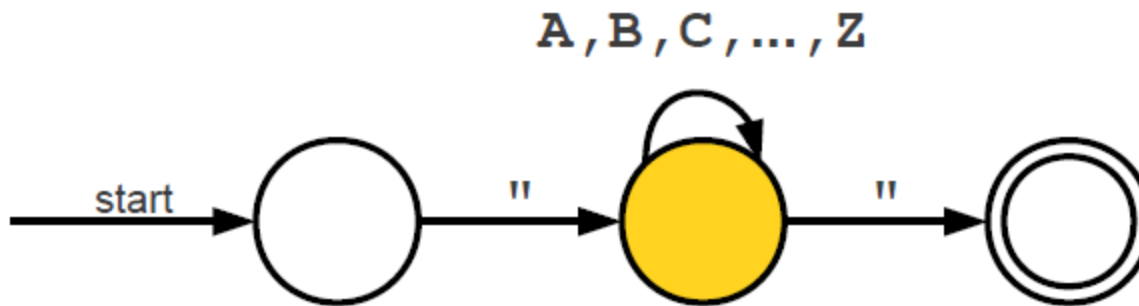
Automat (ví dụ)



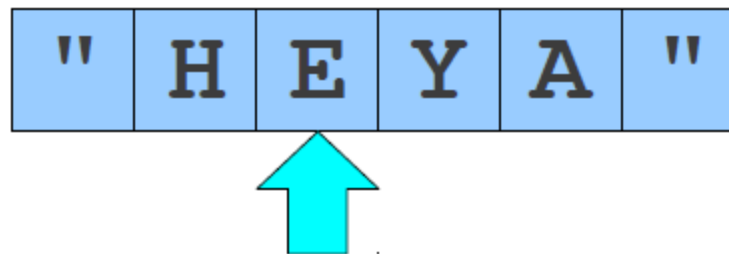
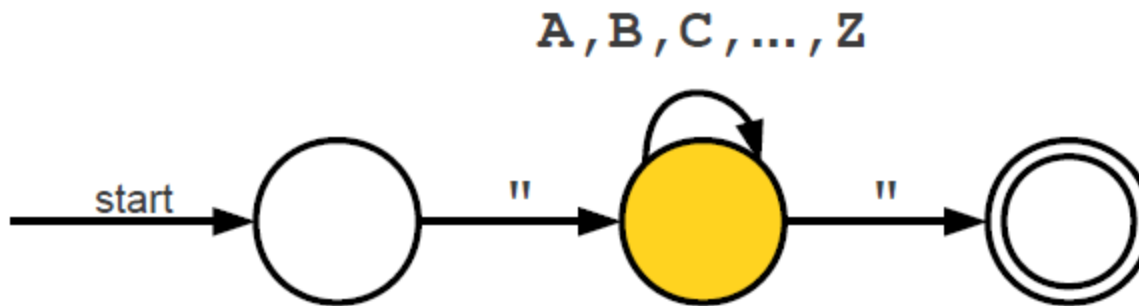
Automat (ví dụ)



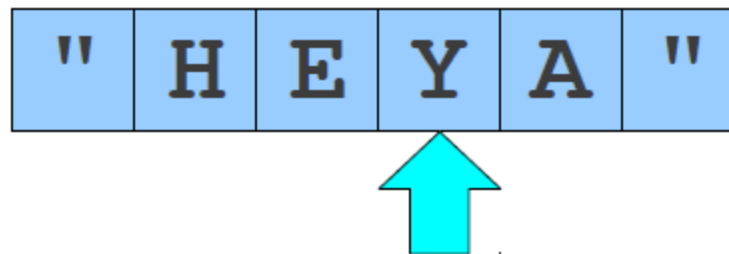
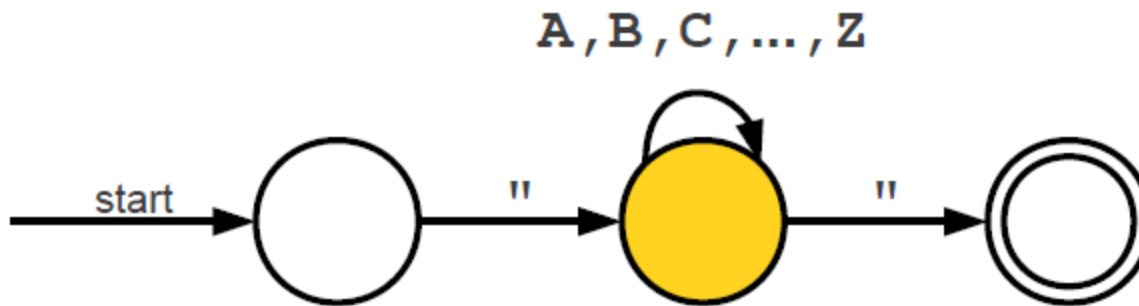
Automat (ví dụ)



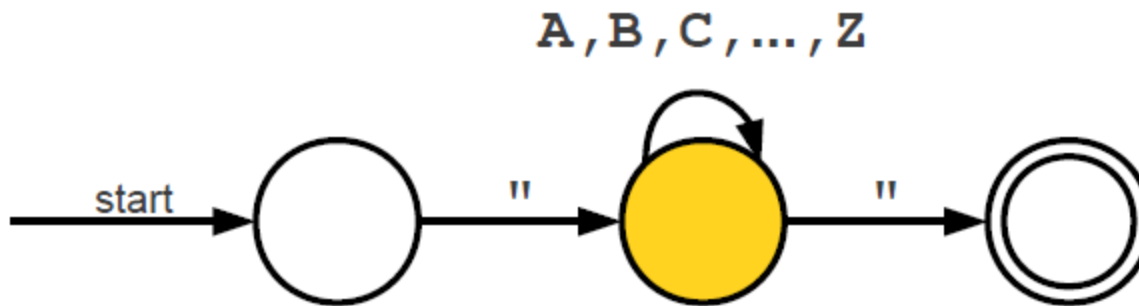
Automat (ví dụ)



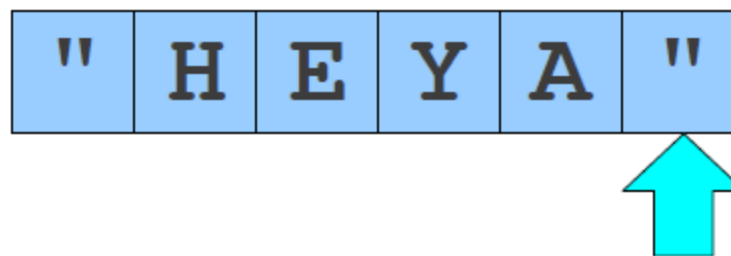
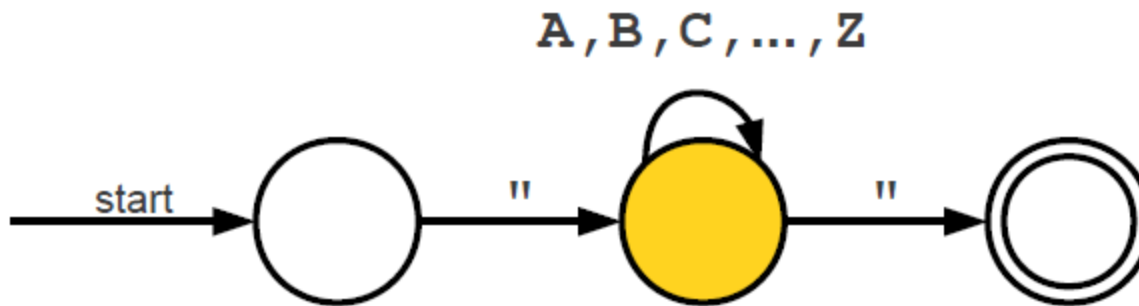
Automat (ví dụ)



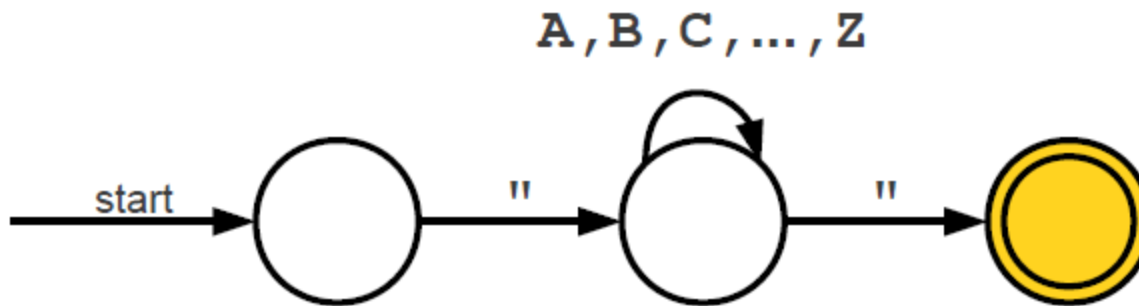
Automat (ví dụ)



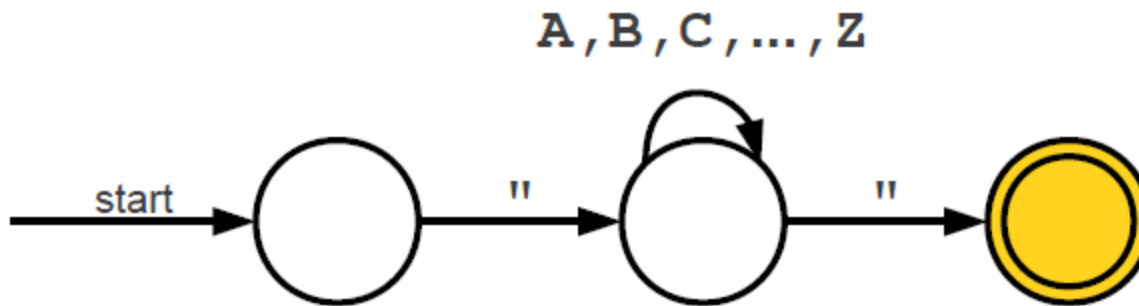
Automat (ví dụ)



Automat (ví dụ)

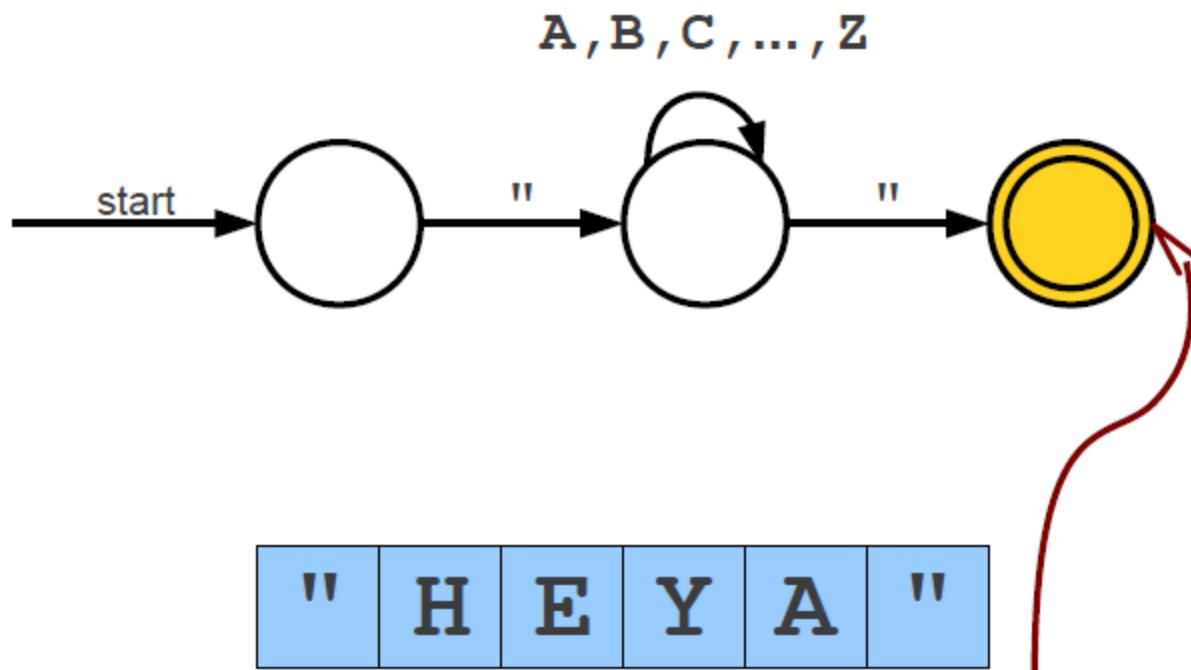


Automat (ví dụ)



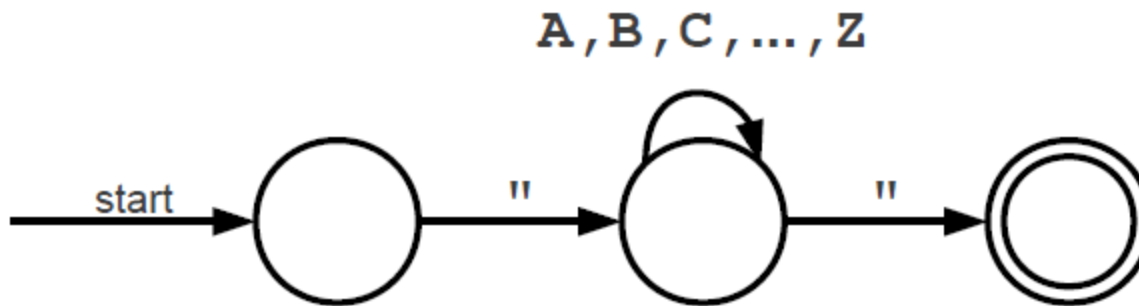
"	H	E	Y	A	"
---	---	---	---	---	---

Automat (ví dụ)

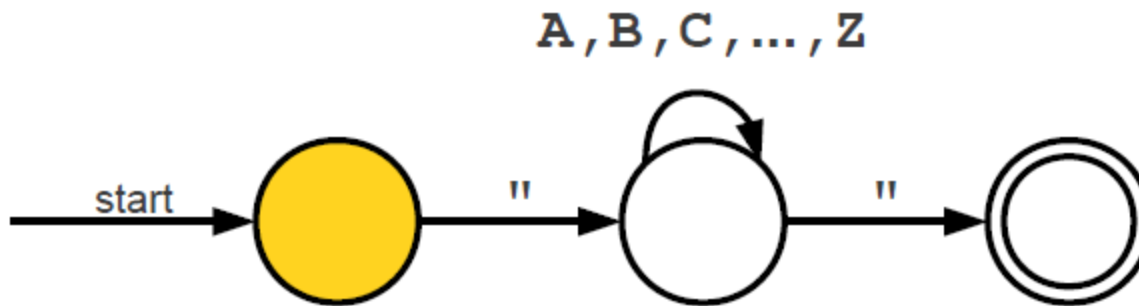


Trạng biểu diễn bằng vòng tròn kép là **trạng thái chấp nhận (accepting state)**.
Automat sẽ chấp nhận chuỗi nhập nếu nó kết thúc tại trạng thái một trong các trạng thái chấp nhận.

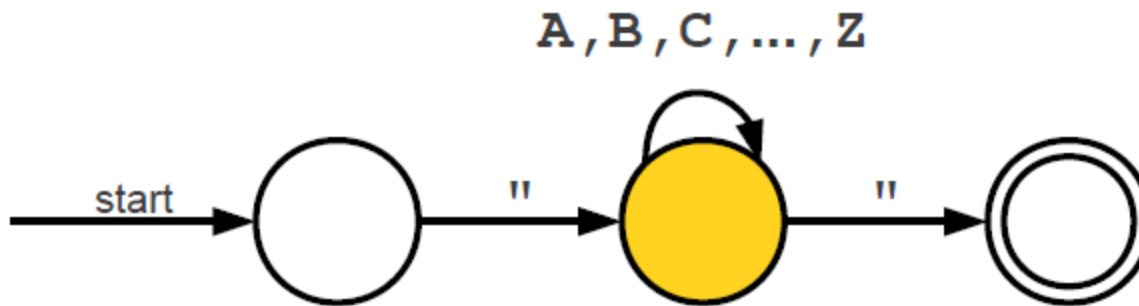
Automat (ví dụ)



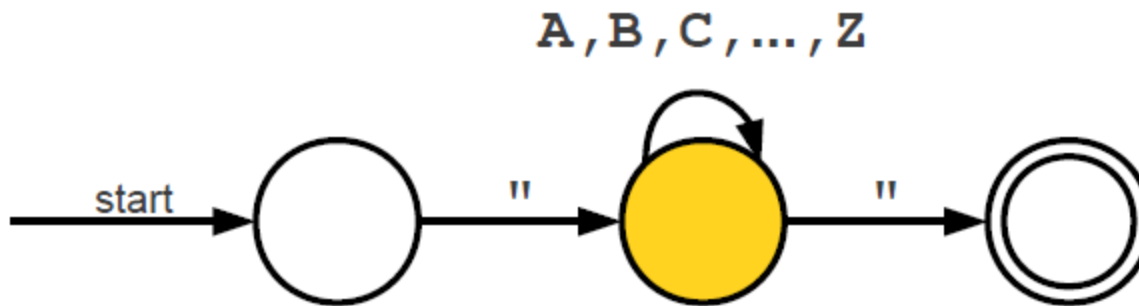
Automat (ví dụ)



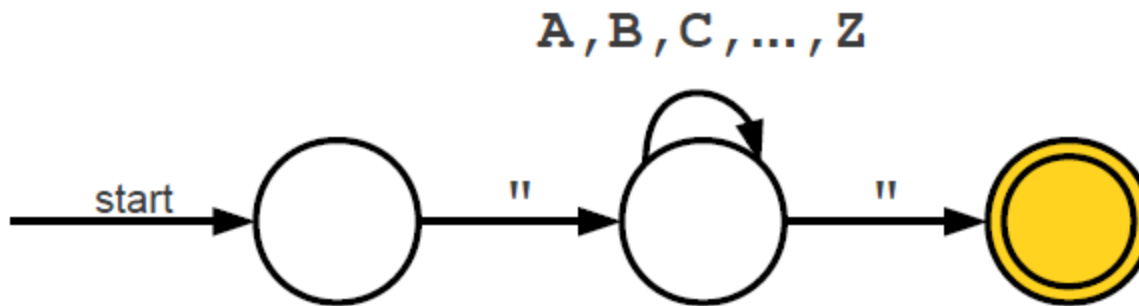
Automat (ví dụ)



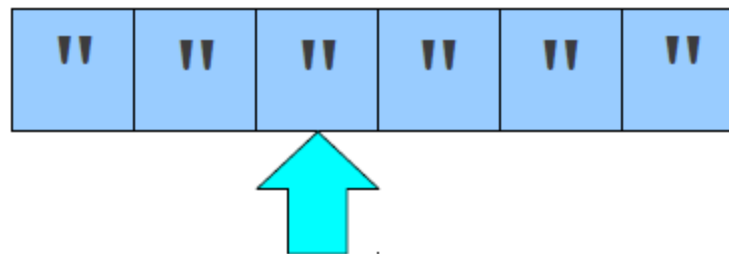
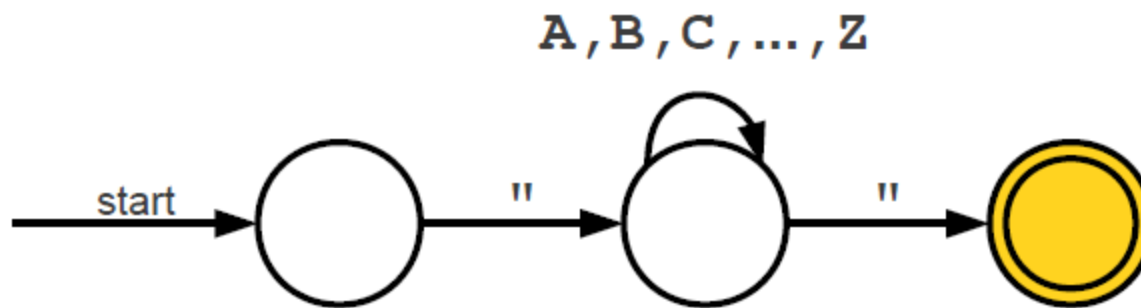
Automat (ví dụ)



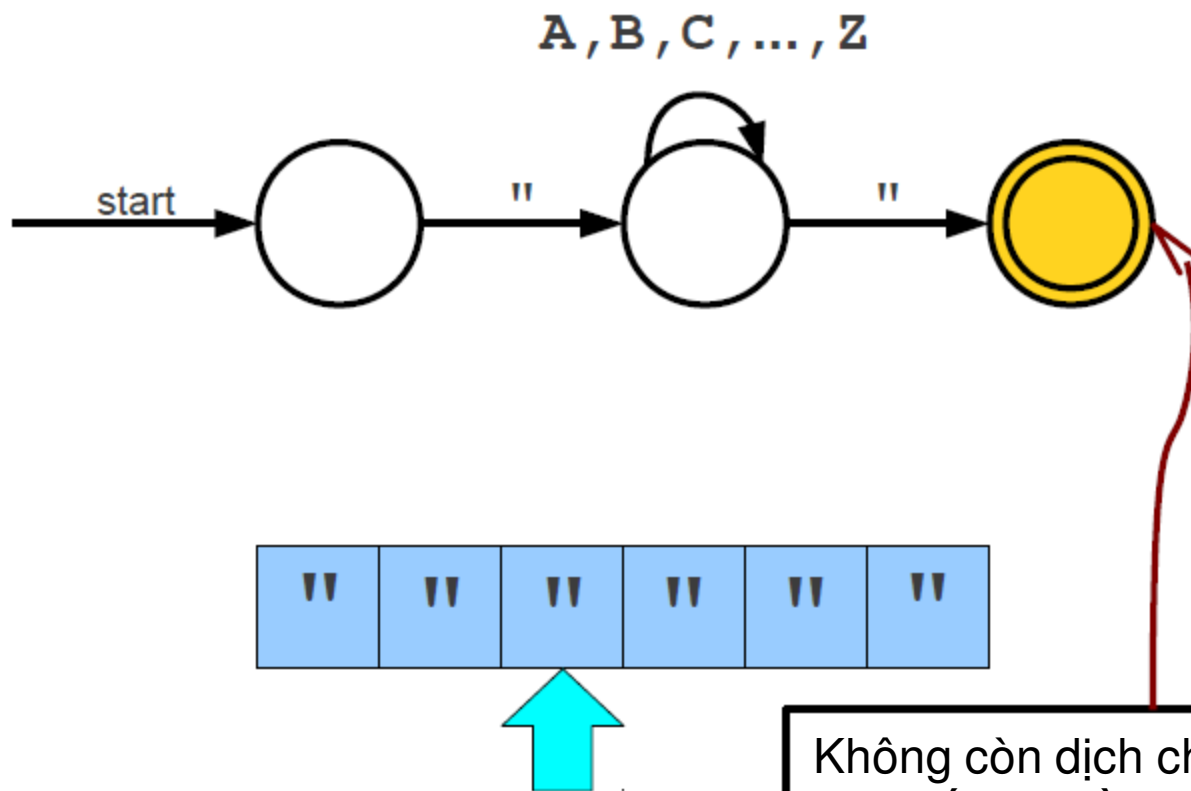
Automat (ví dụ)



Automat (ví dụ)

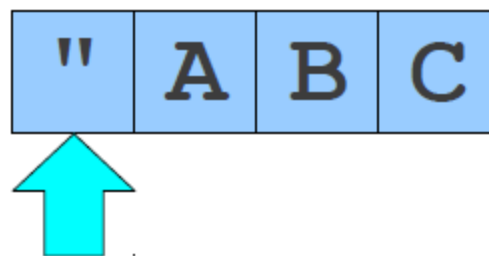
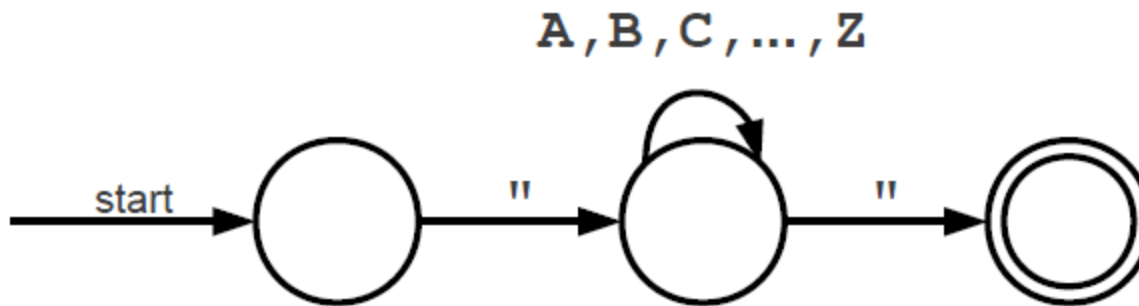


Automat (ví dụ)

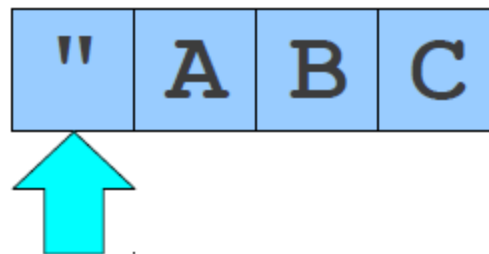
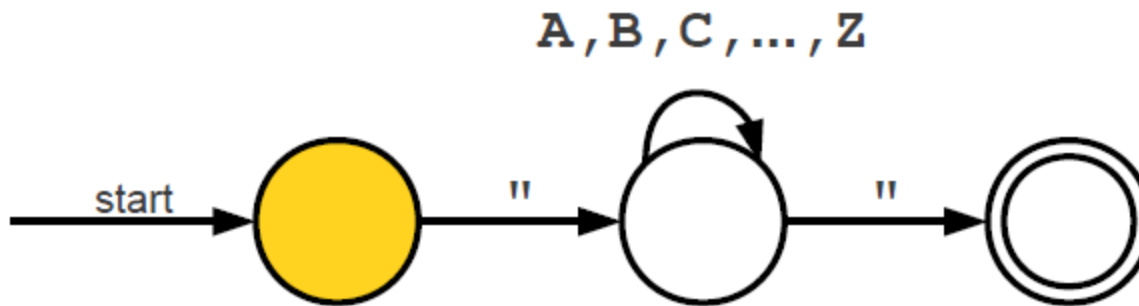


Không còn dịch chuyển nào nữa đối với đầu vào " và trạng thái hiện hành, automat **chết (die)** và bác bỏ đầu vào.

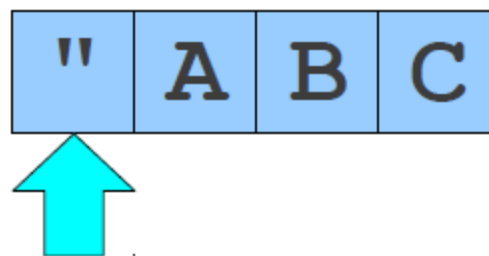
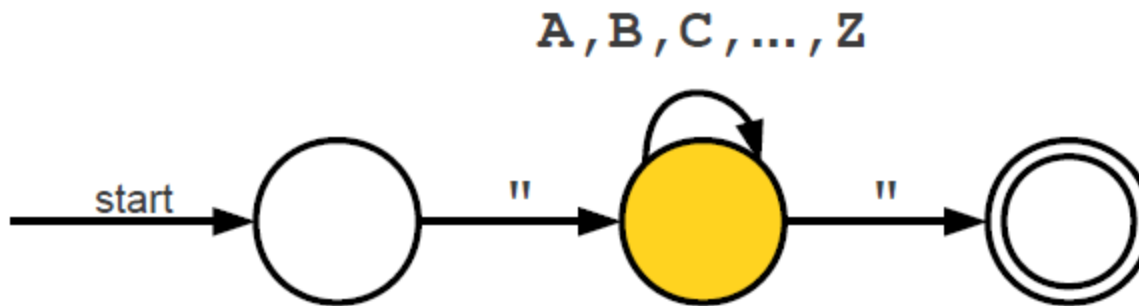
Automat (ví dụ)



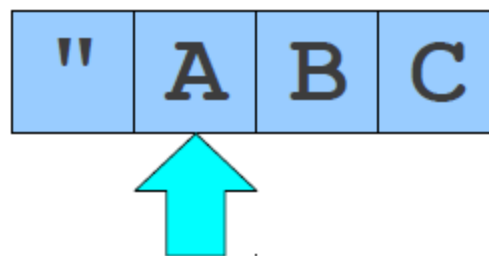
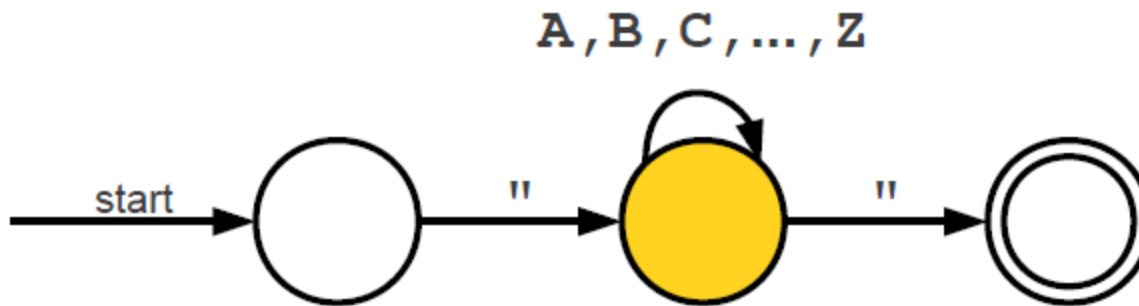
Automat (ví dụ)



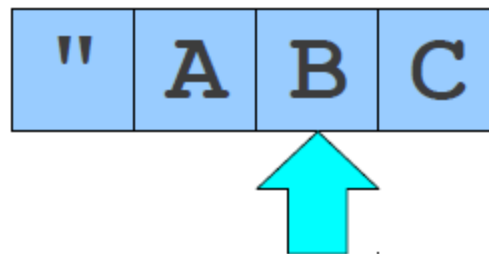
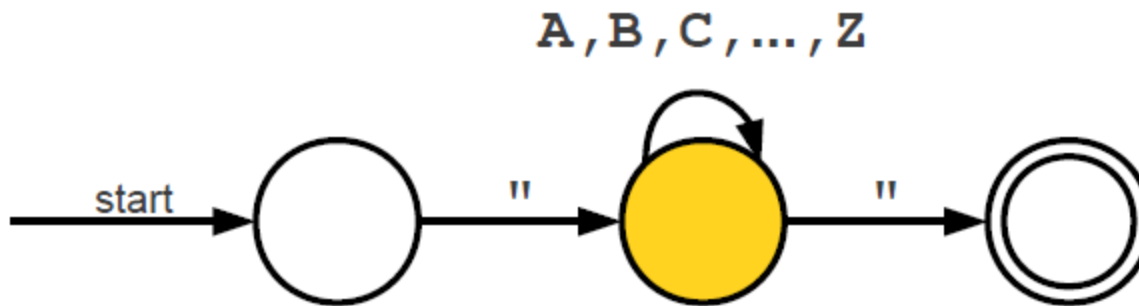
Automat (ví dụ)



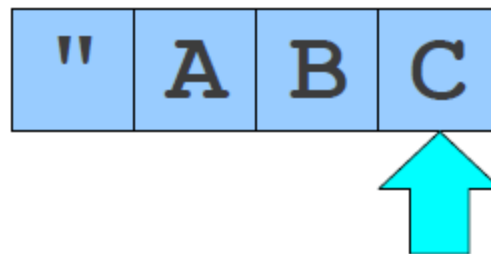
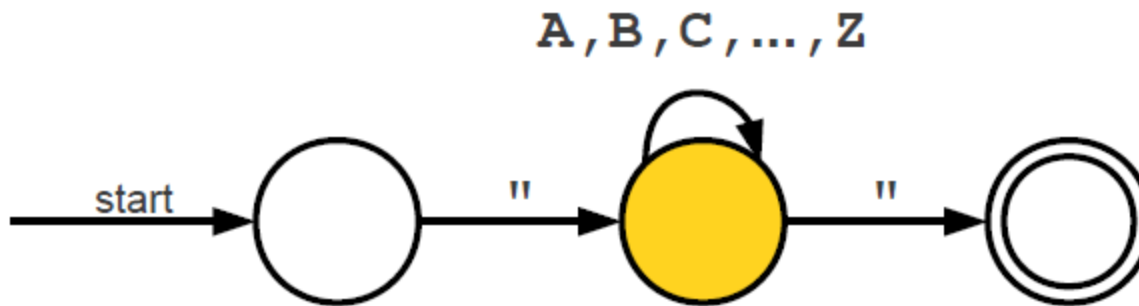
Automat (ví dụ)



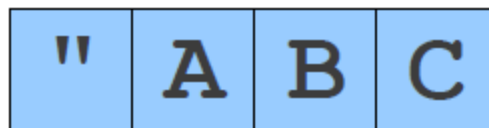
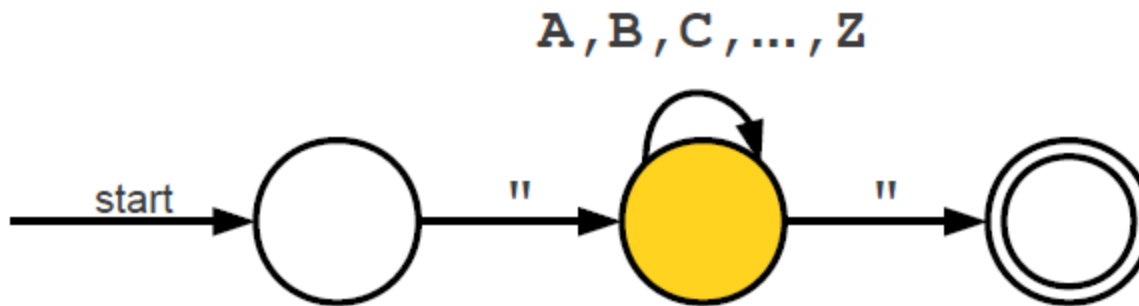
Automat (ví dụ)



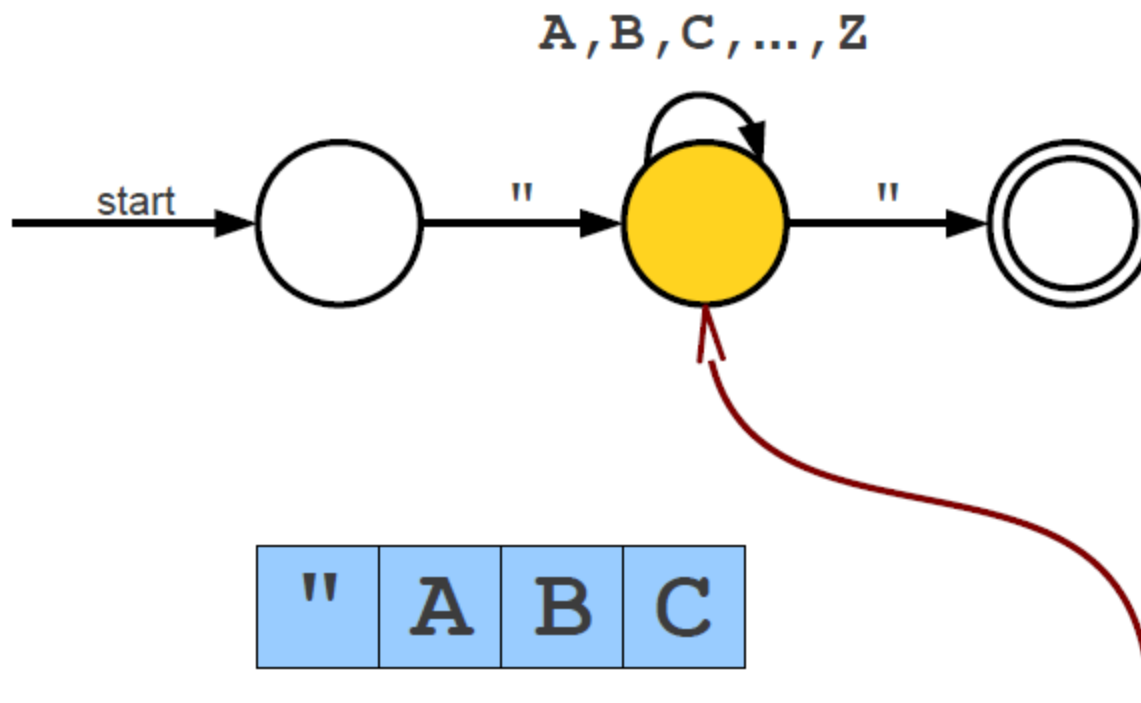
Automat (ví dụ)



Automat (ví dụ)



Automat (ví dụ)



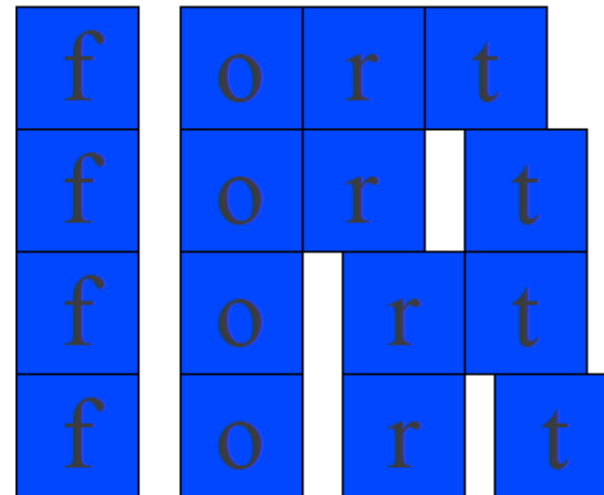
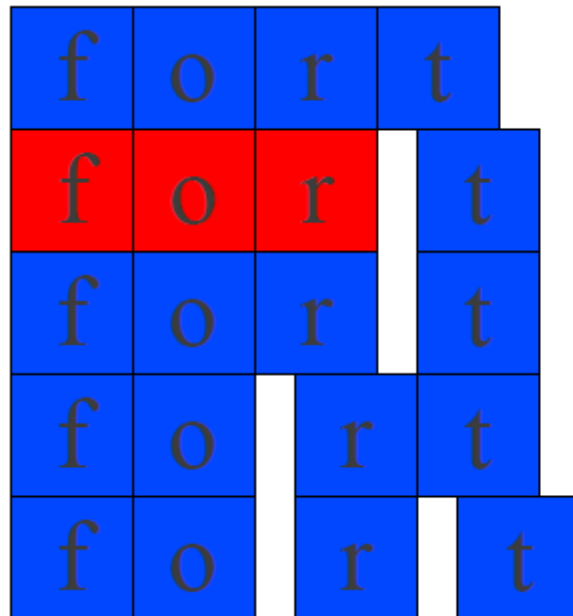
Đây không phải trạng thái chấp nhận, vì vậy automat bác bỏ đầu vào.

Sự mơ hồ (Ambiguities) trong phân tích từ vựng

- T_For for
- T_Identifier [A-Za-z][A-Za-z0-9]*

- Với đầu vào:

f	o	r	t
---	---	---	---



Giải quyết độn độ

- Giả sử tất cả các token được mô tả bằng các biểu thức chính quy.
- Giải thuật: duyệt từ trái sang phải (**Left-to-right scan**)
- Luật tách: tiêu thụ nhiều nhất có thể (**Maximal munch**)
 - Luôn chọn chuỗi dài nhất

Giải quyết độ

- T_For for
- T_Identifier [A-Za-z][A-Za-z0-9]*

- Với đầu vào:

f	o	r	t
---	---	---	---

f	o	r	t
f	o	r	t
f	o	r	t
f	o	r	t
f	o	r	t

f	o	r	t
f	o	r	t
f	o	r	t
f	o	r	t

f	o	r	t
---	---	---	---

Cài đặt giải thuật Maximal munch

- Cho tập các biểu thức chính quy, làm thế nào sử dụng chúng để cài đặt giải thuật **Maximal munch** ?
- Ý tưởng:
 - Xây dựng các automat từ các biểu thức chính quy.
 - Chạy automat song song và giữ lại **lần khớp sau cùng (last match)**.
 - Khi tất cả các automat không chạy được nữa, trả về lần khớp sau cùng và quay trở lại điểm đó.

Cài đặt giải thuật Maximal munch

T_Do	do
T_Double	double
T_Mystery	[A-Za-z]

Cài đặt giải thuật Maximal munch

T_Do

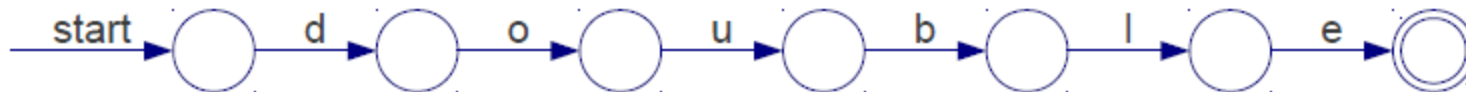
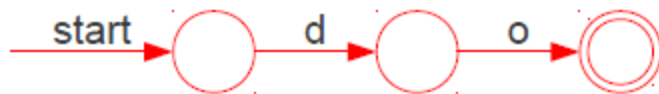
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

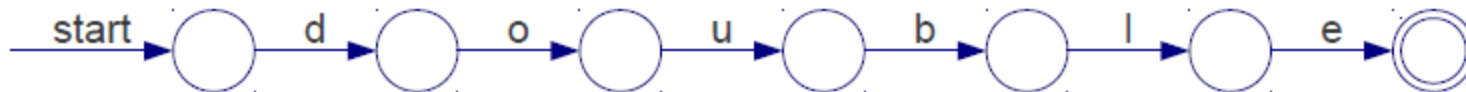
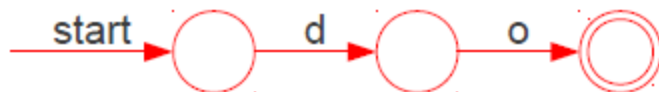
do

T_Double

double

T_Mystery

[A-Za-z]



D	O	U	B	D	O	U	B	L	E
---	---	---	---	---	---	---	---	---	---

Cài đặt giải thuật Maximal munch

T_Do

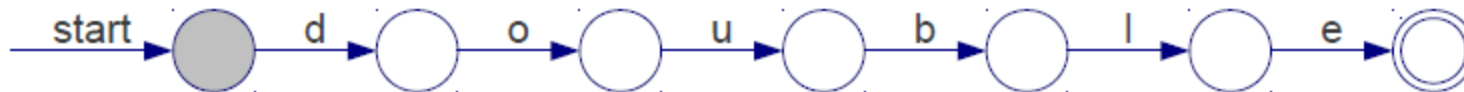
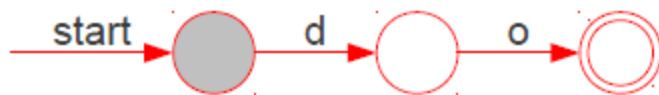
do

T_Double

double

T_Mystery

[A-Za-z]



D	O	U	B	D	O	U	B	L	E
---	---	---	---	---	---	---	---	---	---

Cài đặt giải thuật Maximal munch

T_Do

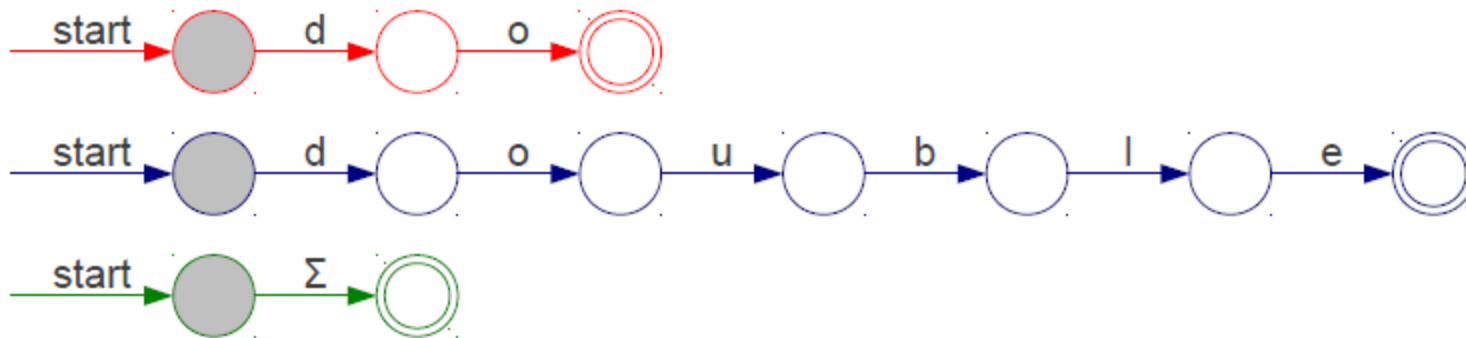
do

T_Double

double

T_Mystery

[A-Za-z]



D	O	U	B	D	O	U	B	L	E
---	---	---	---	---	---	---	---	---	---



Cài đặt giải thuật Maximal munch

T_Do

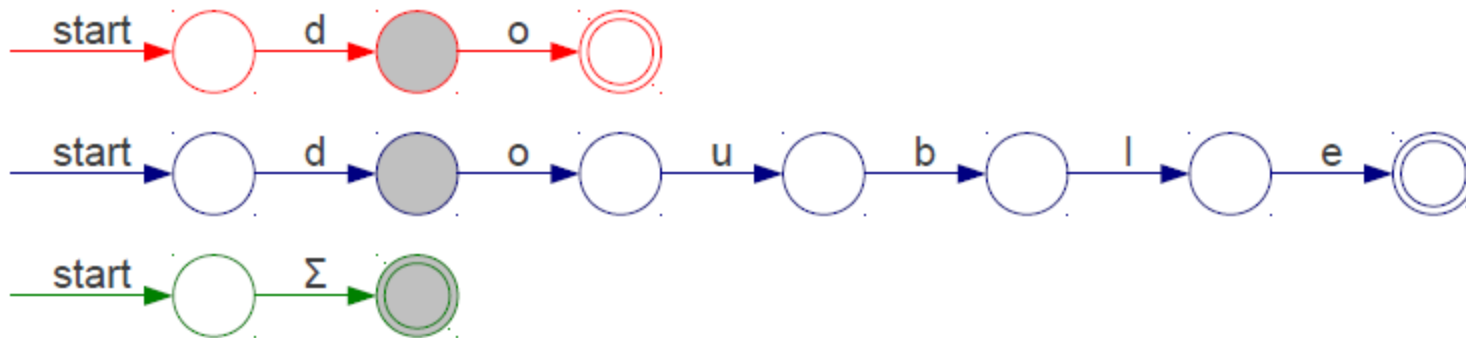
do

T_Double

double

T_Mystery

[A-Za-z]



D	O	U	B	D	O	U	B	L	E
---	---	---	---	---	---	---	---	---	---



Cài đặt giải thuật Maximal munch

T_Do

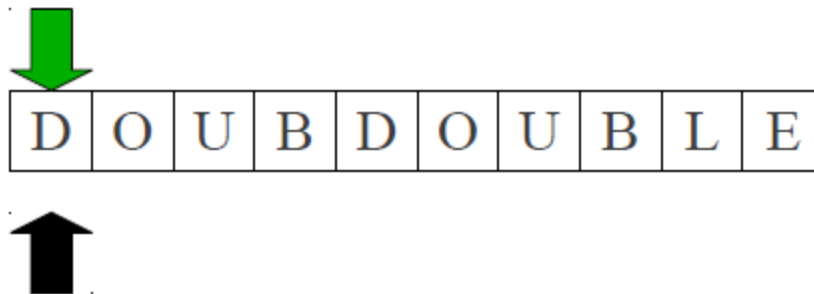
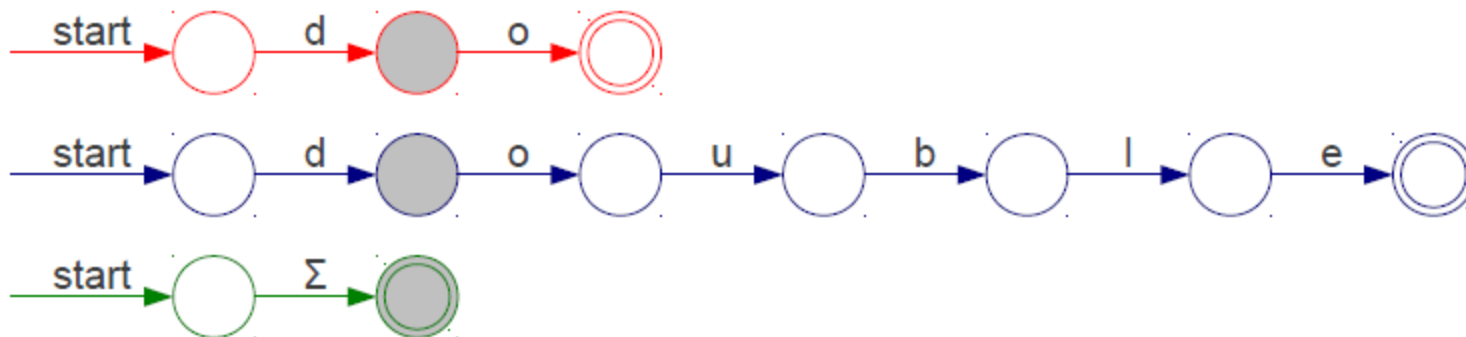
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

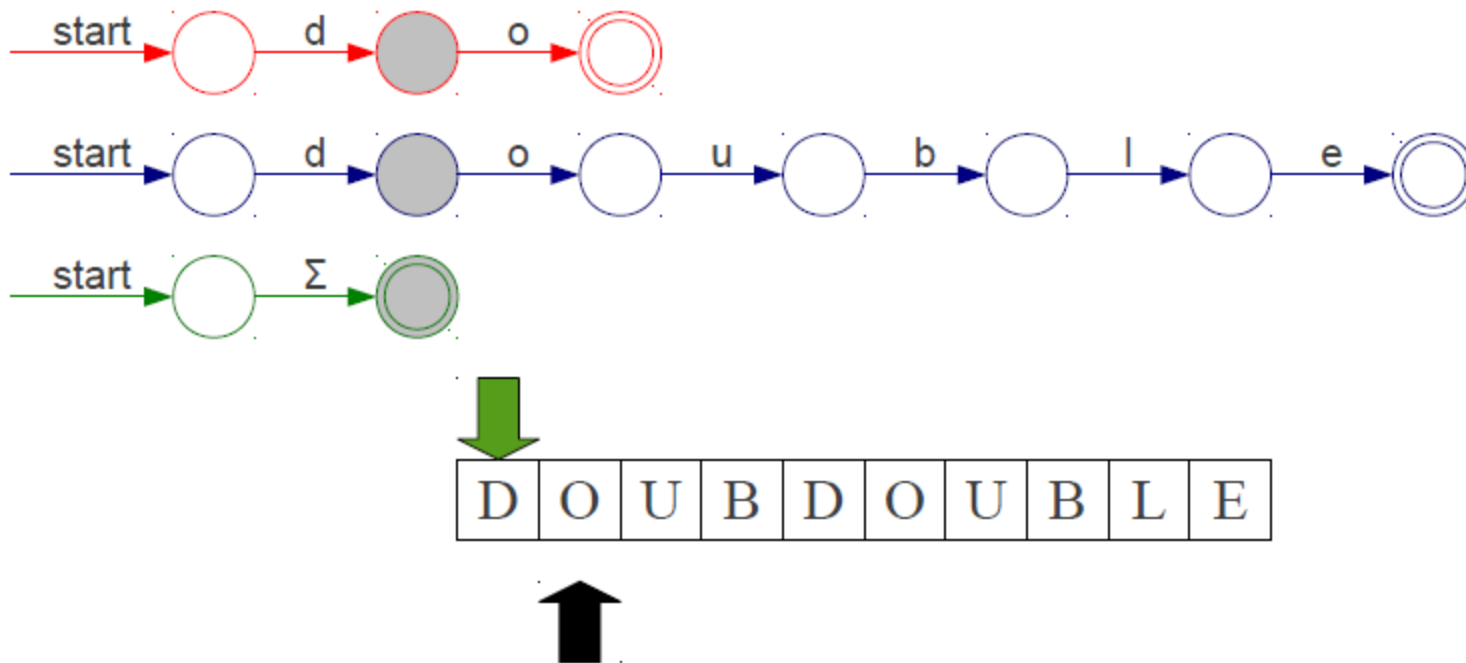
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

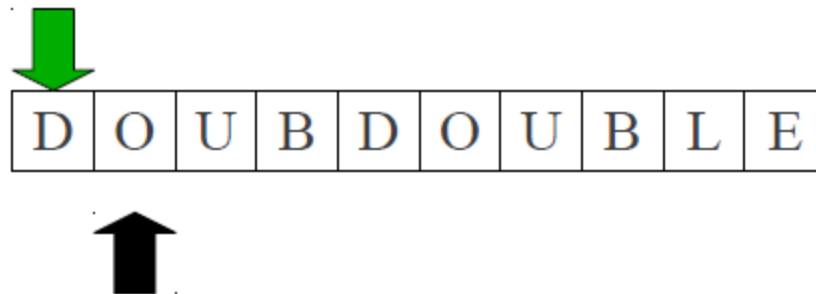
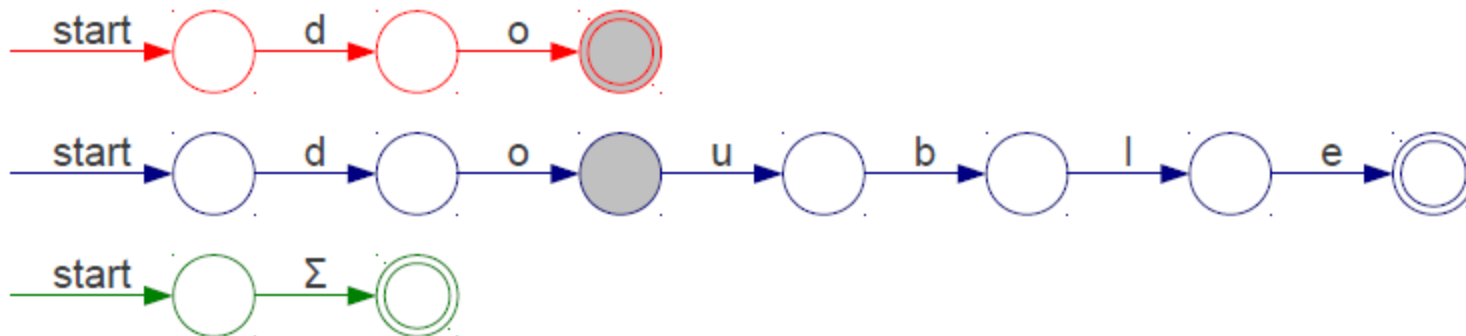
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

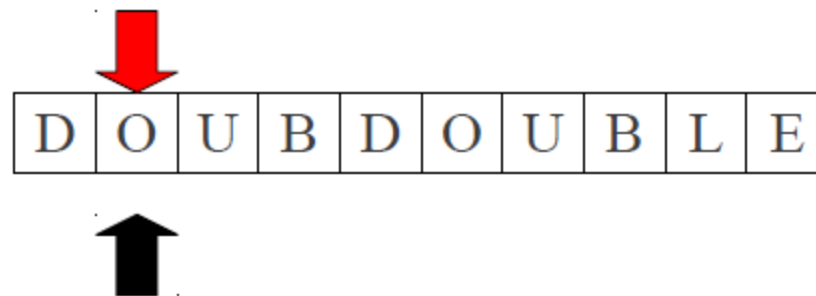
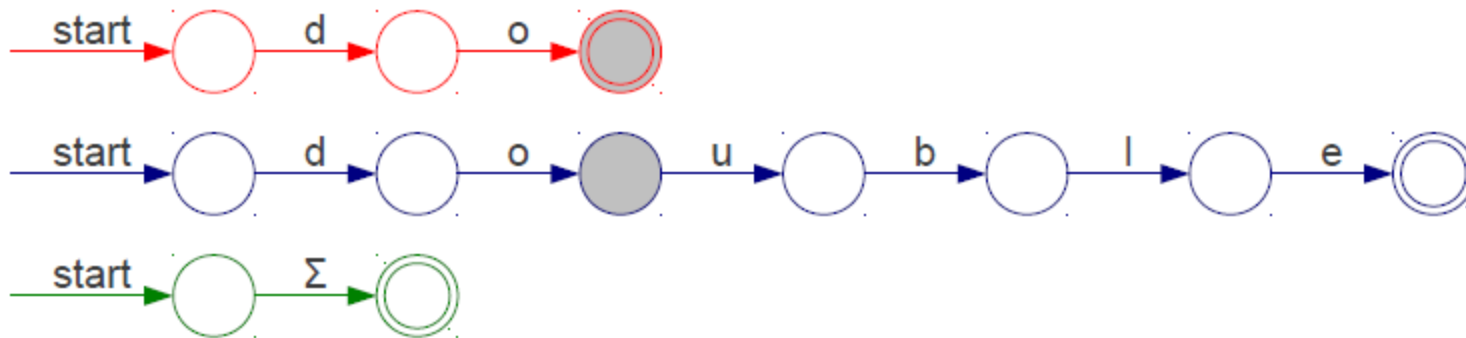
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

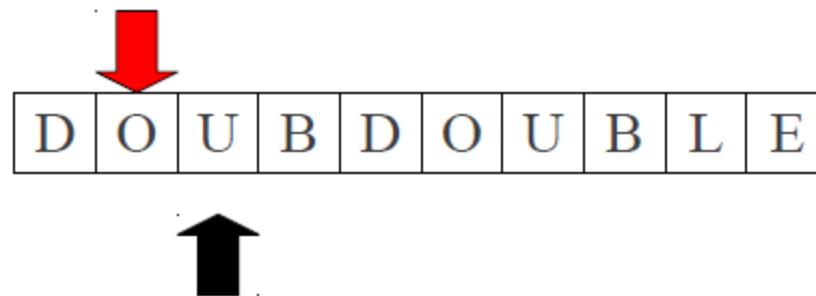
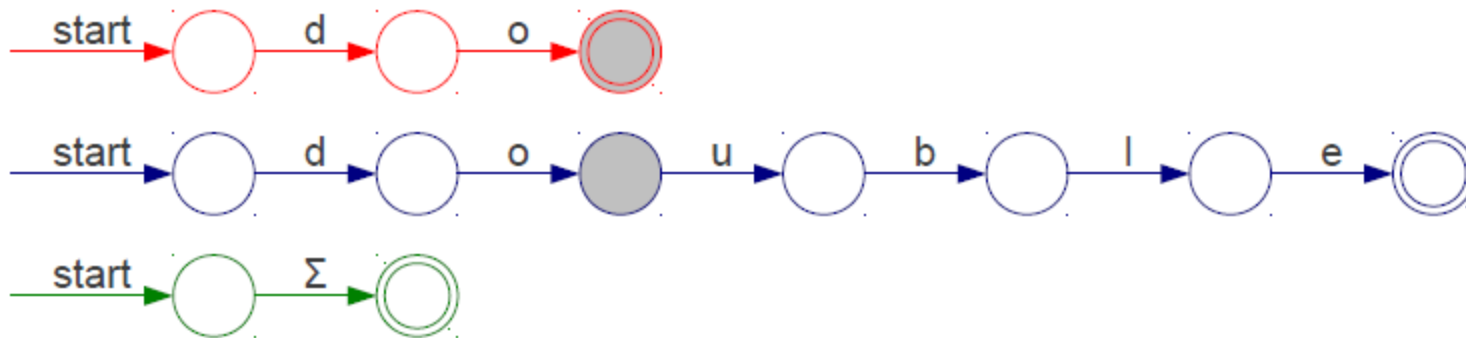
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

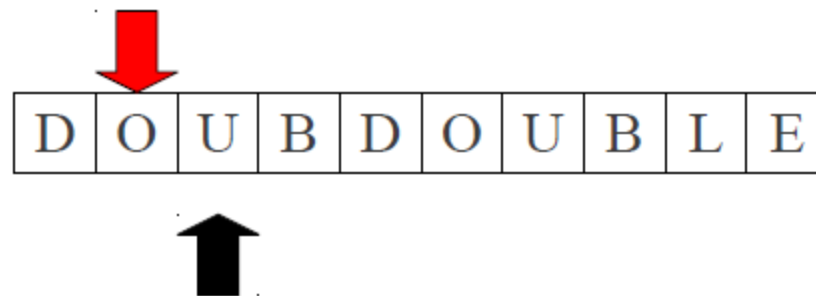
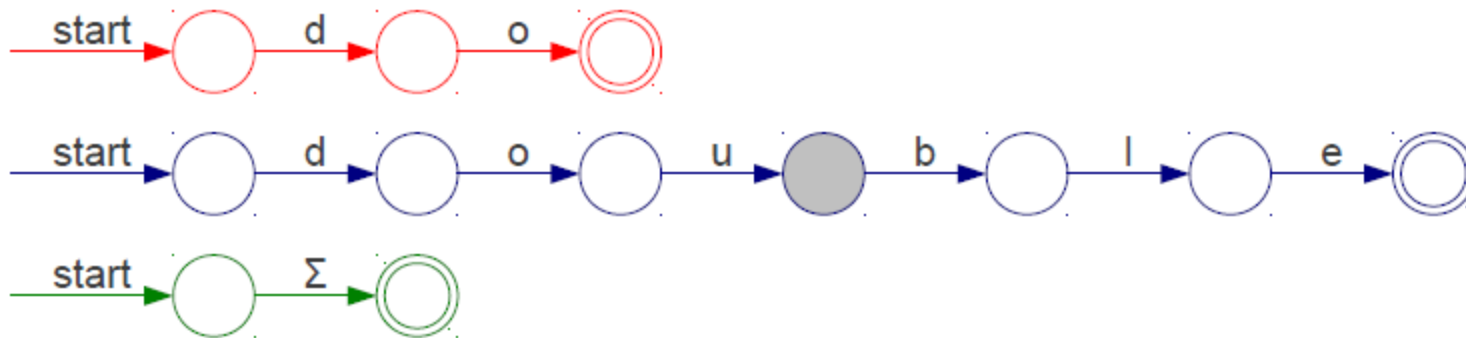
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

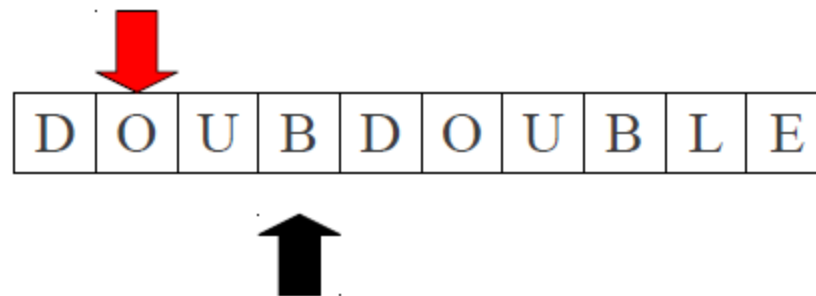
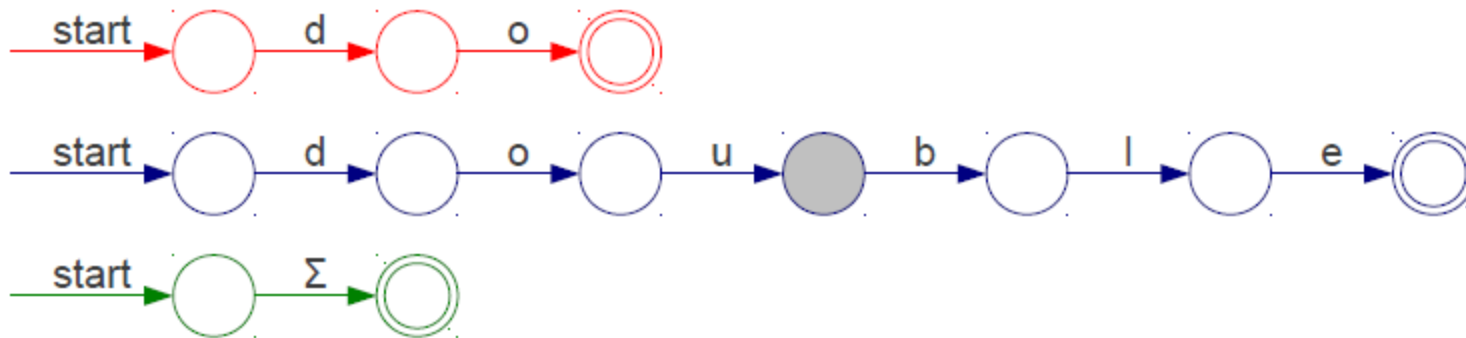
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

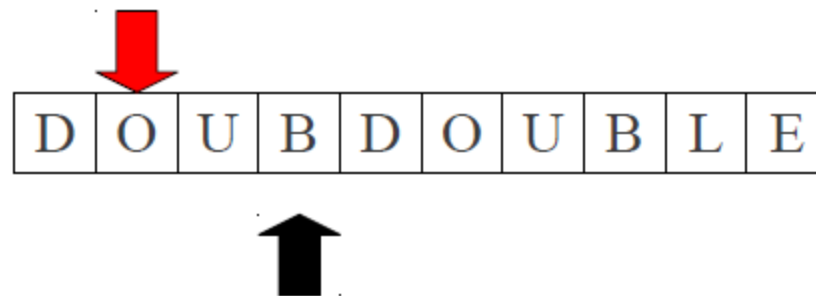
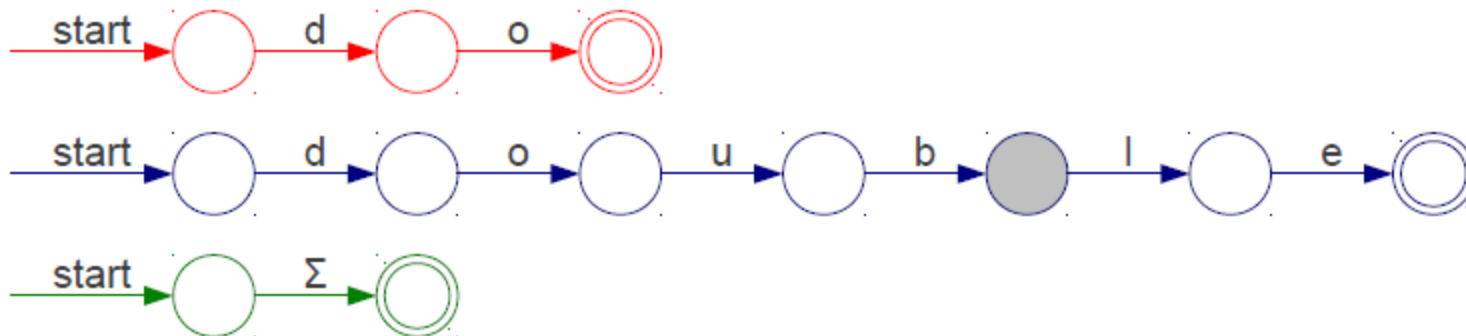
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

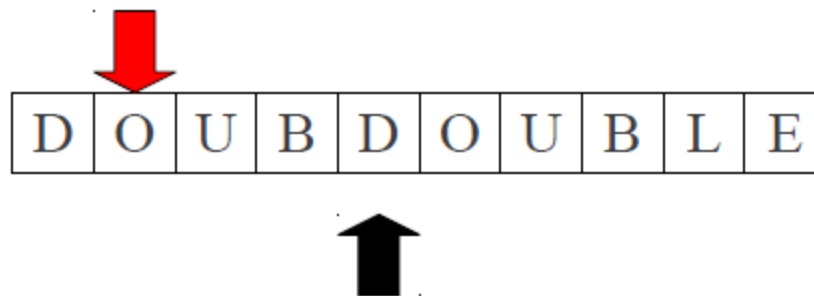
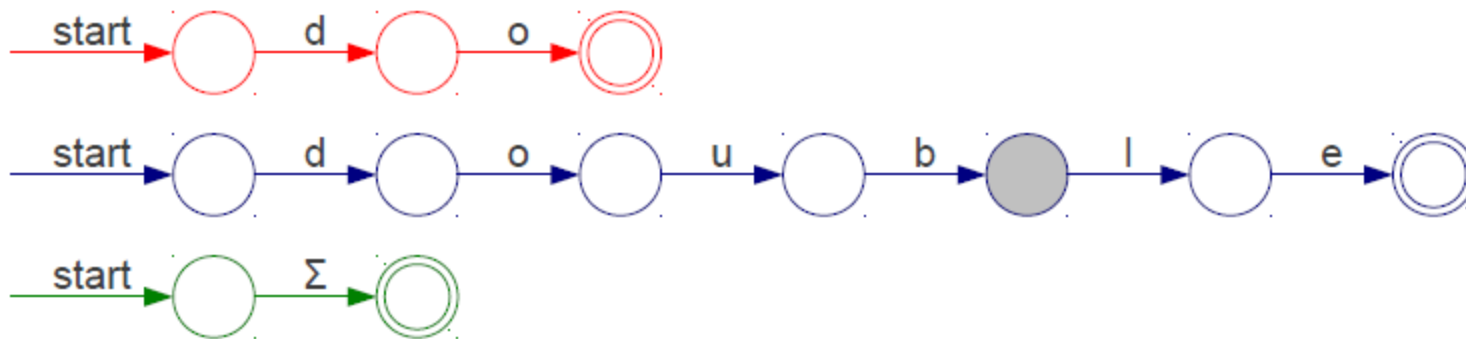
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

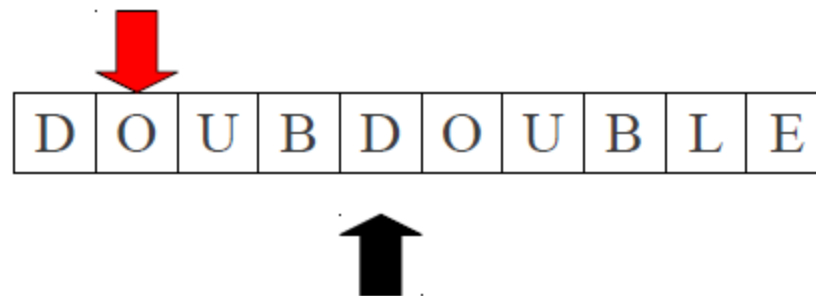
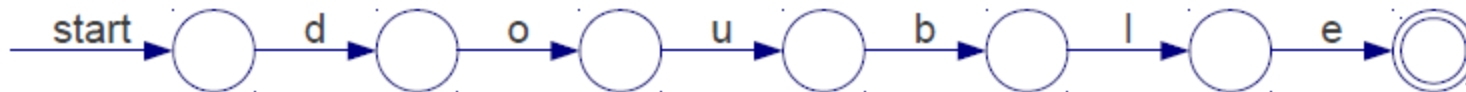
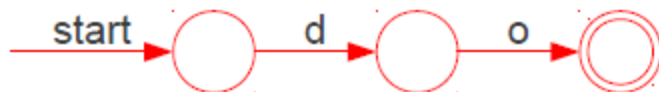
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

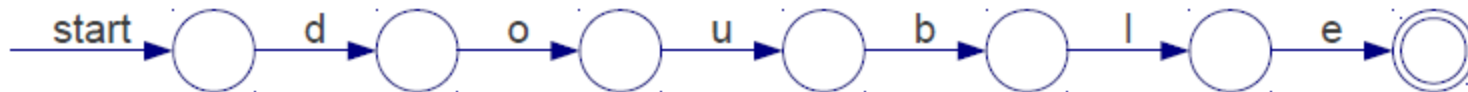
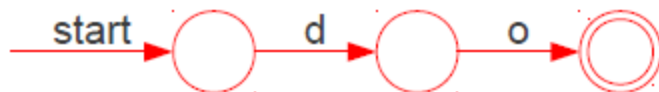
do

T_Double

double

T_Mystery

[A-Za-z]



DO

U B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

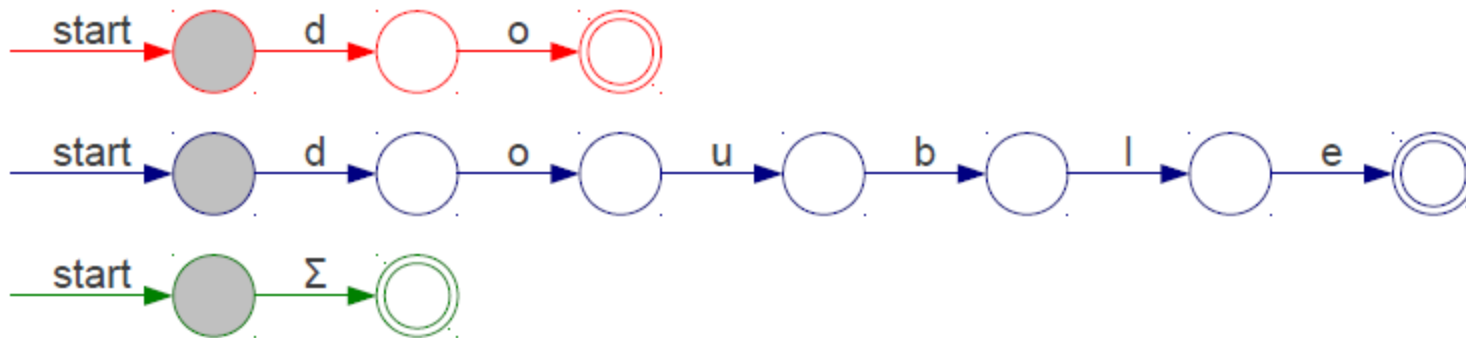
do

T_Double

double

T_Mystery

[A-Za-z]



D O

U B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

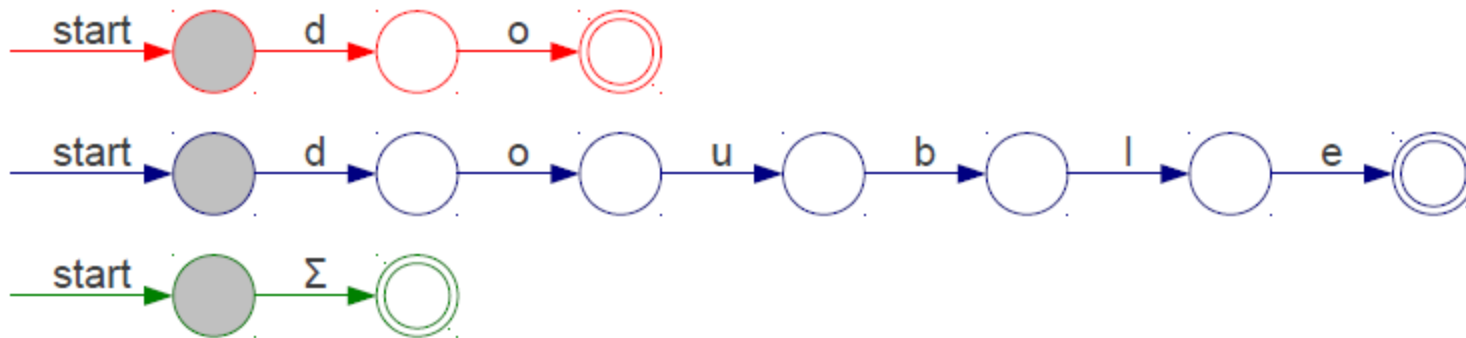
do

T_Double

double

T_Mystery

[A-Za-z]



D O

U B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

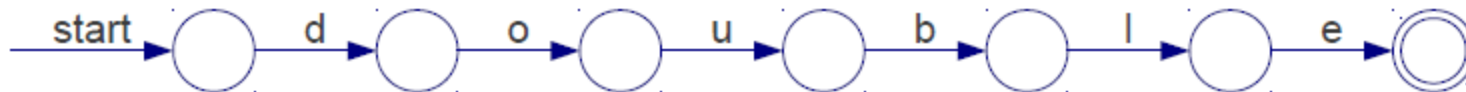
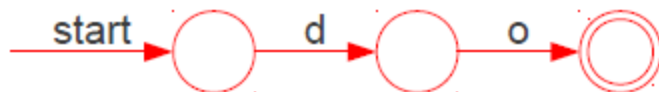
do

T_Double

double

T_Mystery

[A-Za-z]



DO

U B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

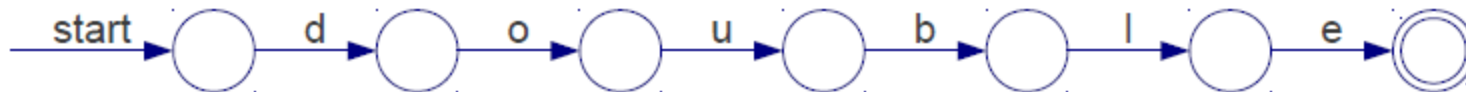
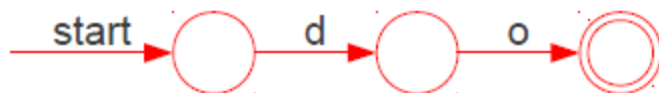
do

T_Double

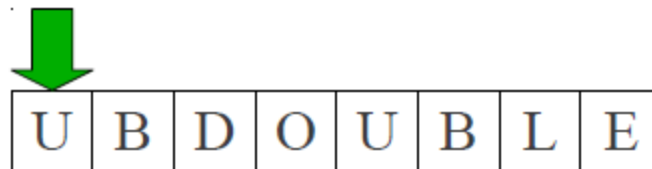
double

T_Mystery

[A-Za-z]



D O



Cài đặt giải thuật Maximal munch

T_Do

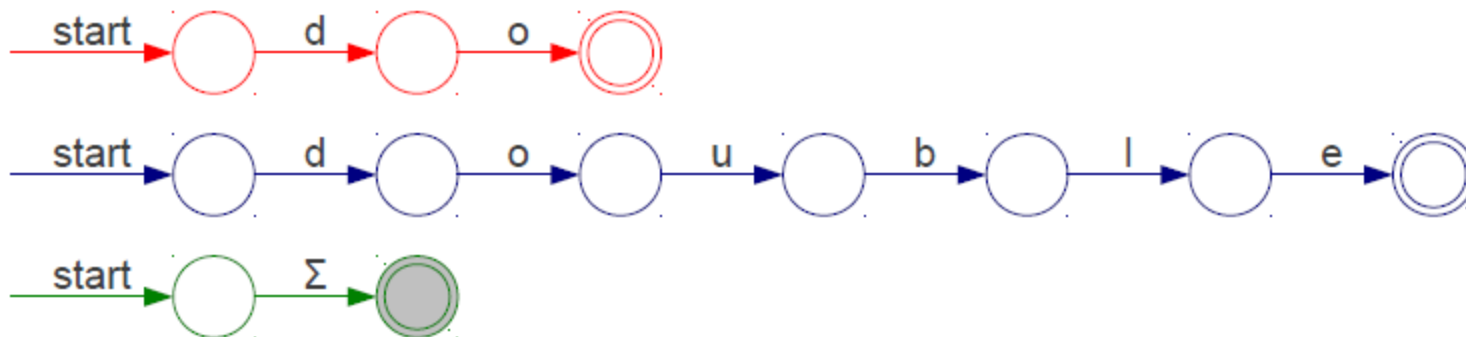
do

T_Double

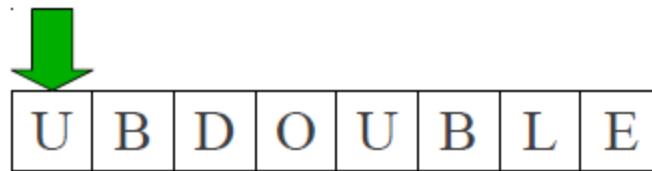
double

T_Mystery

[A-Za-z]



D O



Cài đặt giải thuật Maximal munch

T_Do

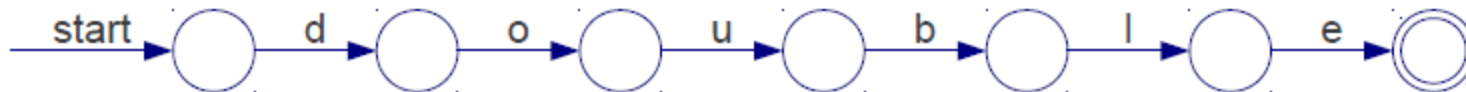
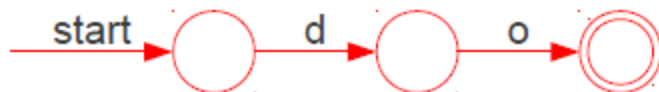
do

T_Double

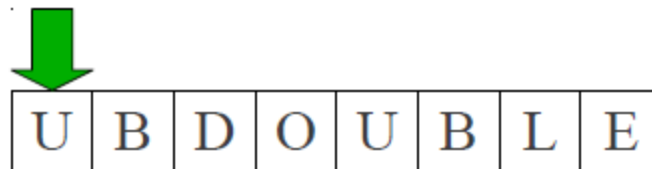
double

T_Mystery

[A-Za-z]



D O



Cài đặt giải thuật Maximal munch

T_Do

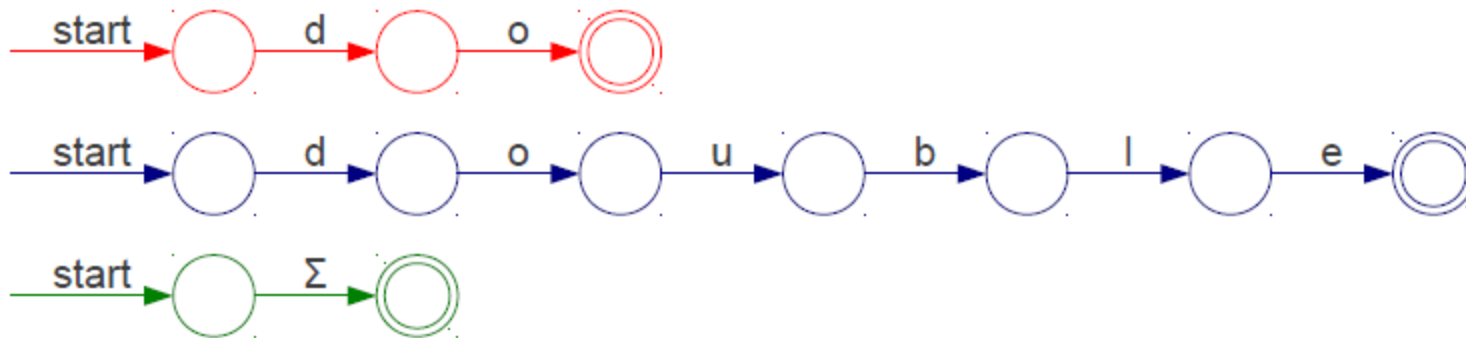
do

T_Double

double

T_Mystery

[A-Za-z]



D O

U

B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

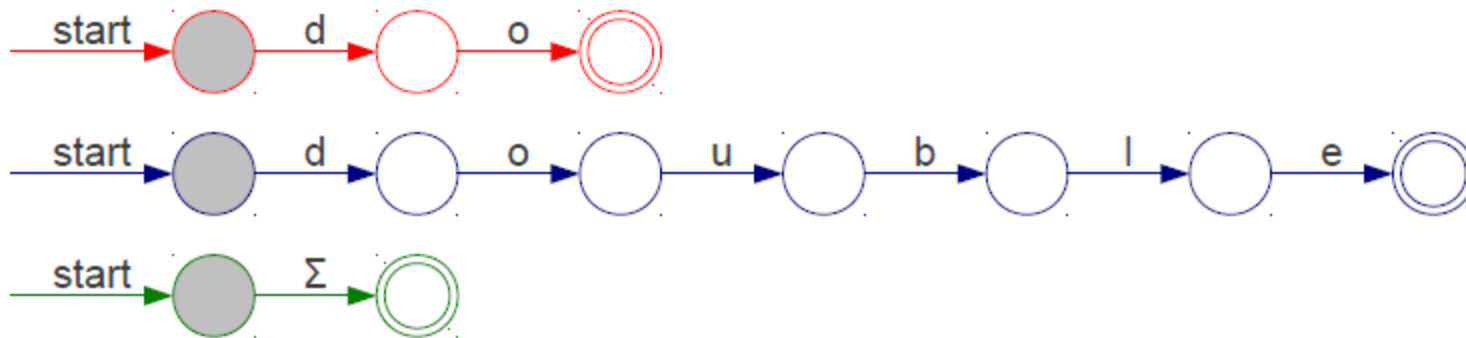
do

T_Double

double

T_Mystery

[A-Za-z]



D O

U

B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

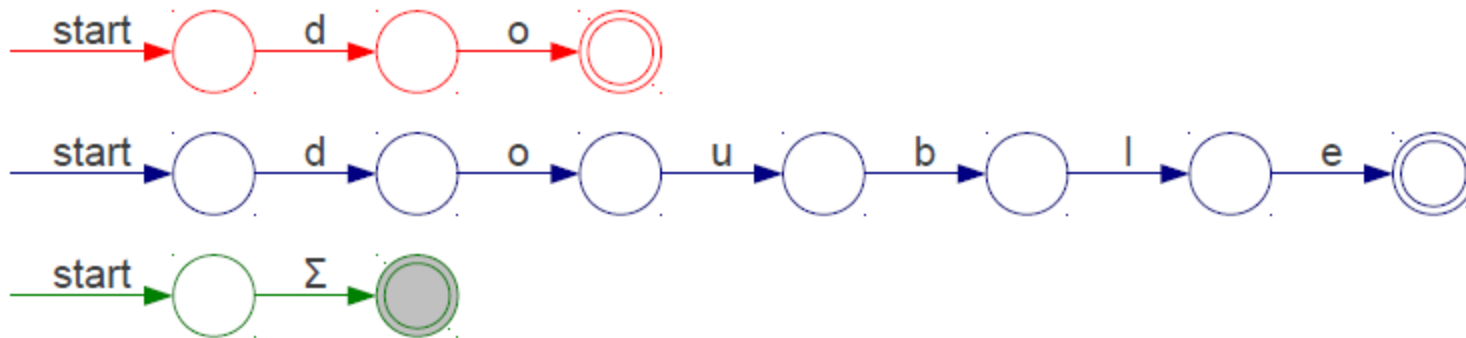
do

T_Double

double

T_Mystery

[A-Za-z]



D O

U

B D O U B L E



Cài đặt giải thuật Maximal munch

T_Do

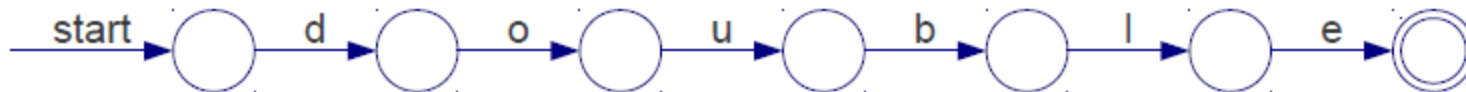
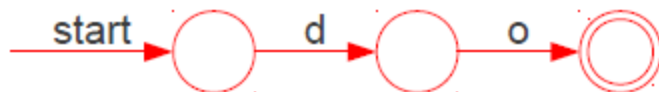
do

T_Double

double

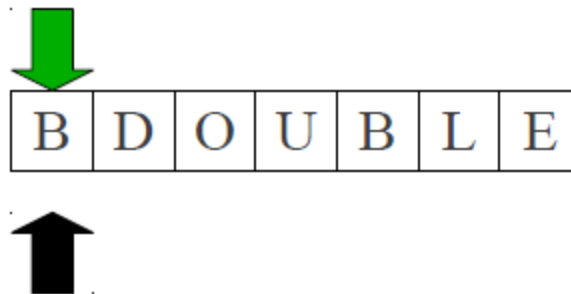
T_Mystery

[A-Za-z]



D O

U



Cài đặt giải thuật Maximal munch

T_Do

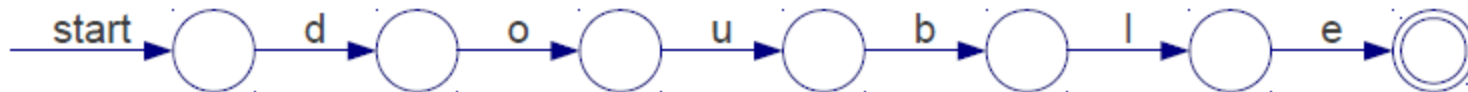
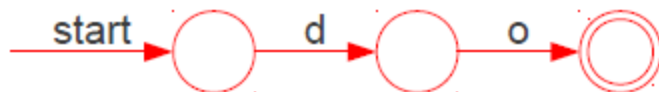
do

T_Double

double

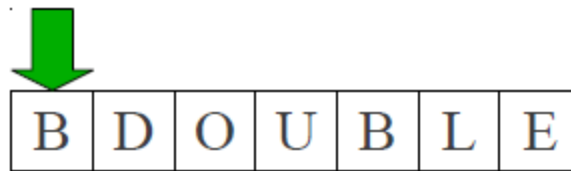
T_Mystery

[A-Za-z]



D O

U



Cài đặt giải thuật Maximal munch

T_Do

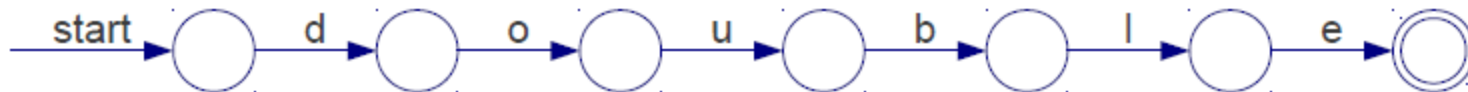
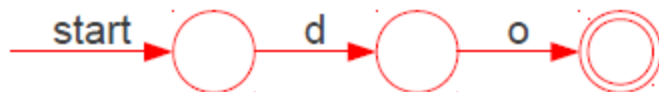
do

T_Double

double

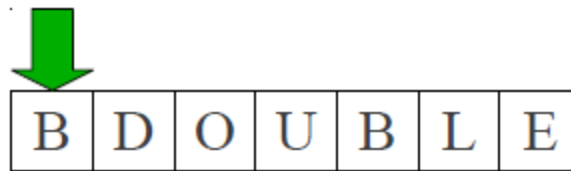
T_Mystery

[A-Za-z]



D O

U



Cài đặt giải thuật Maximal munch

T_Do

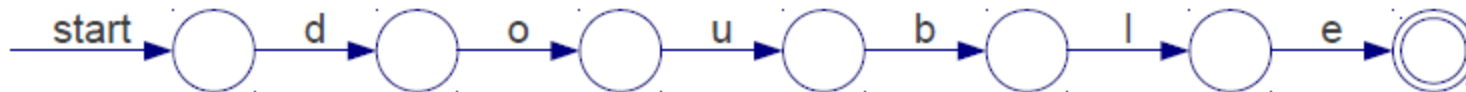
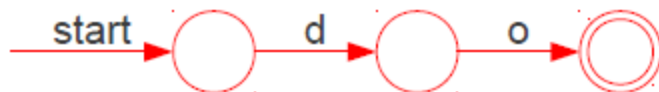
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

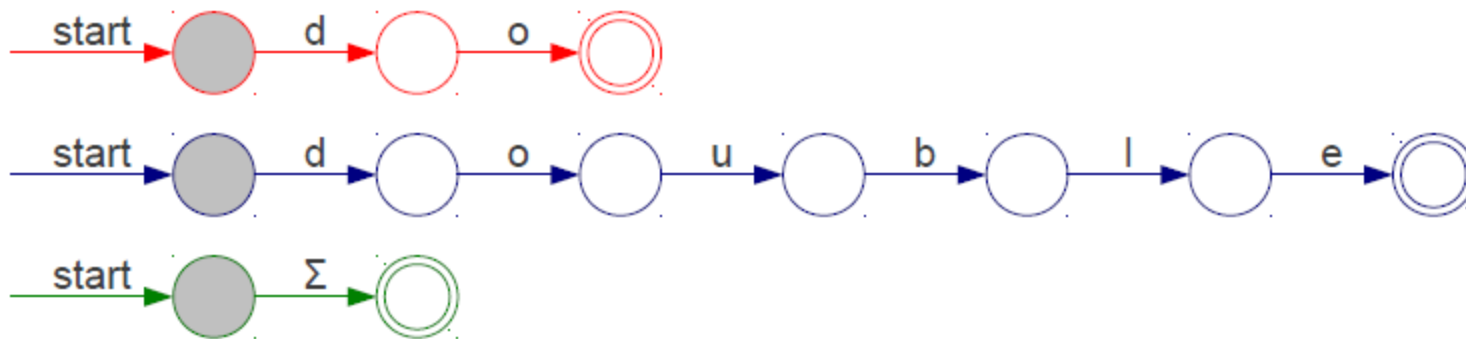
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

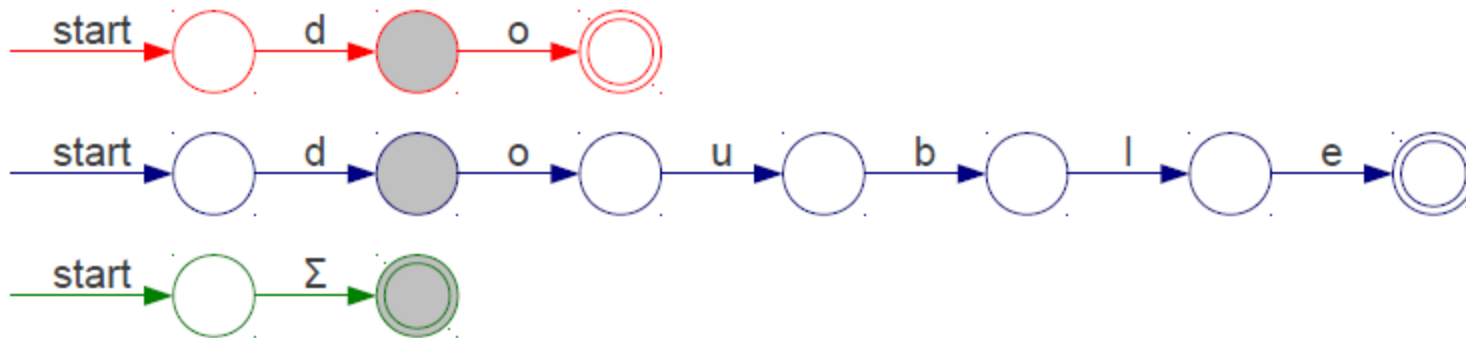
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

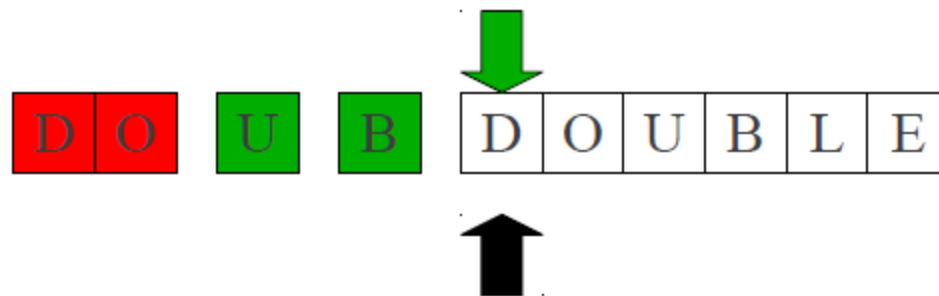
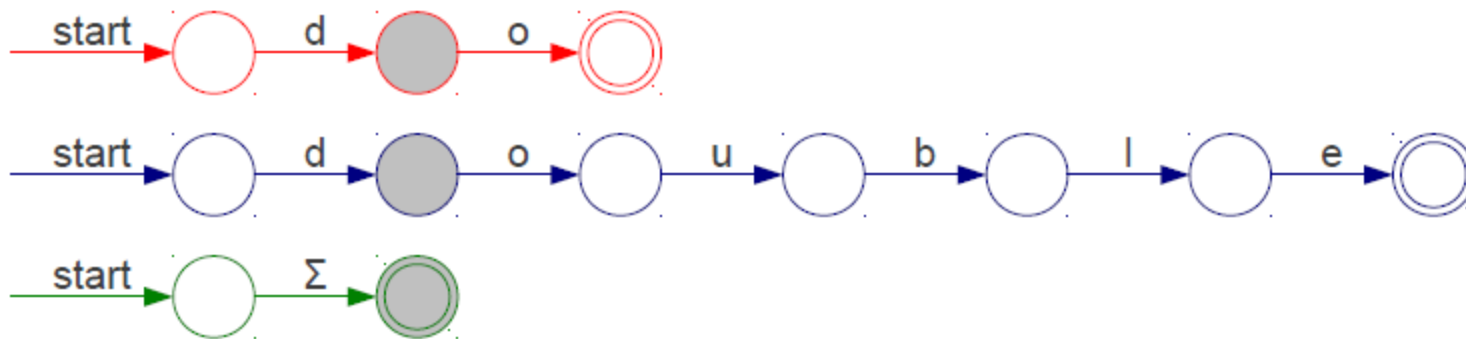
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

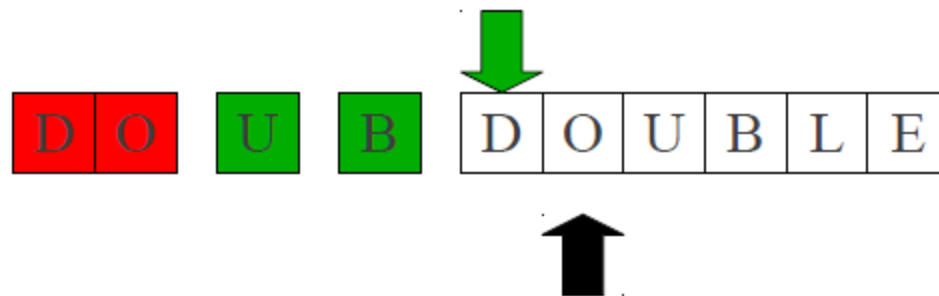
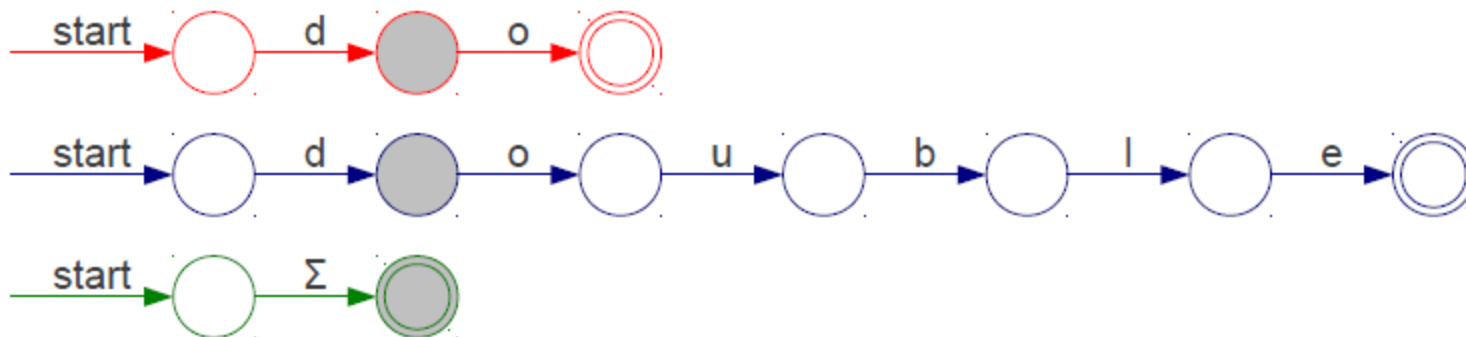
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

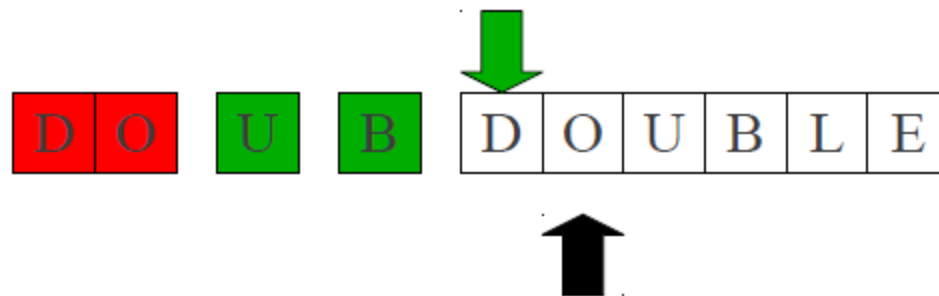
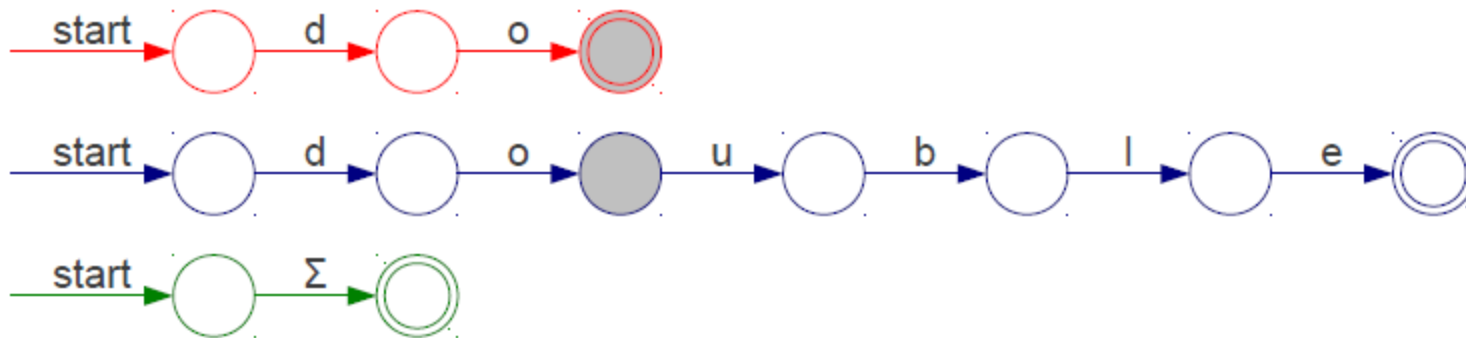
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

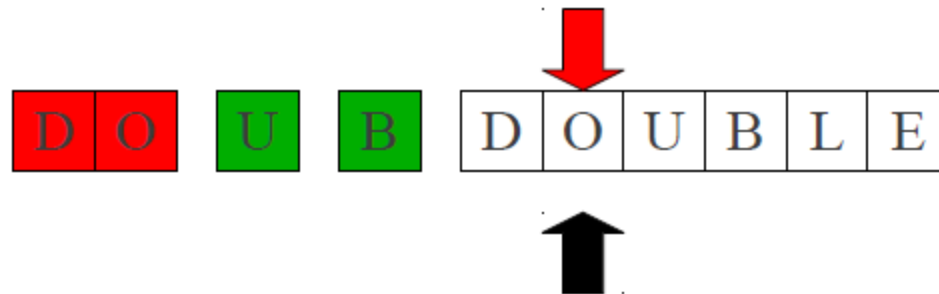
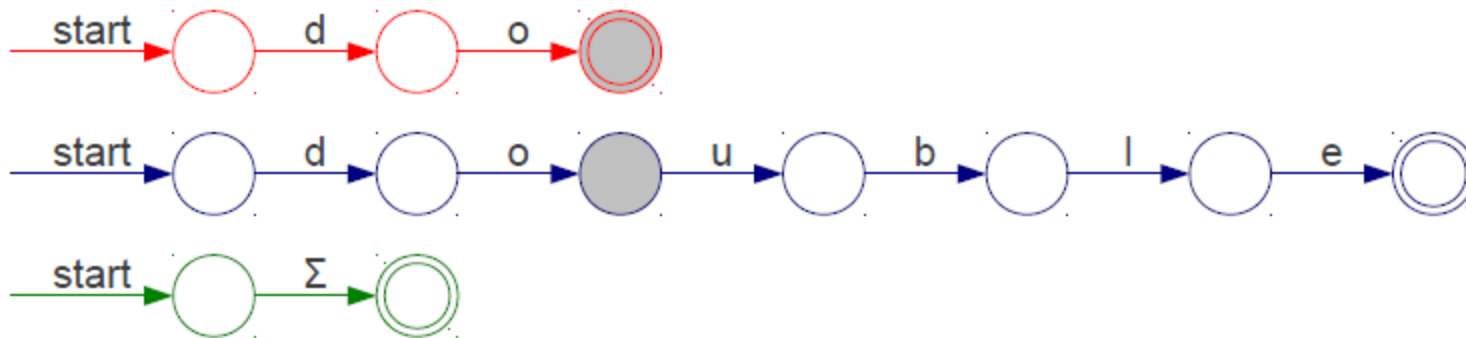
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

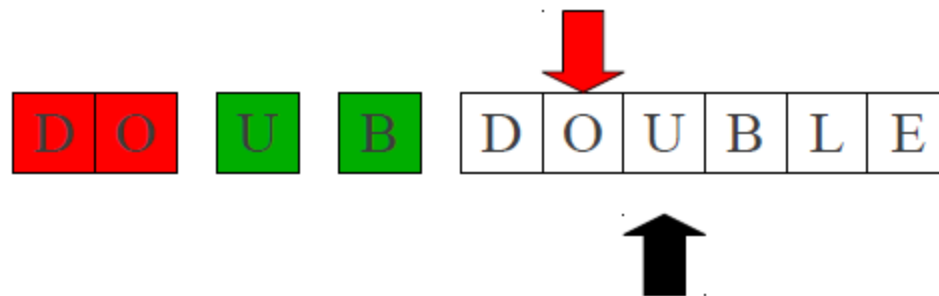
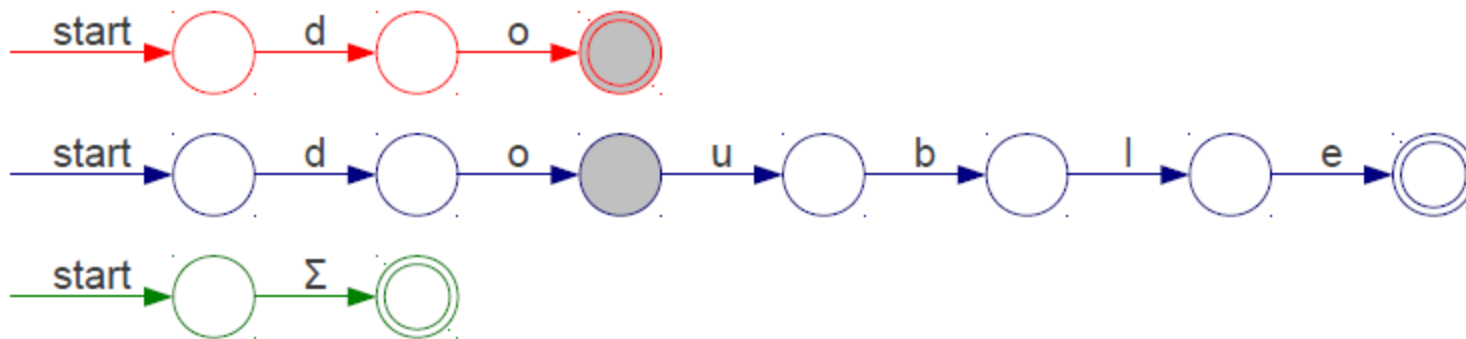
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

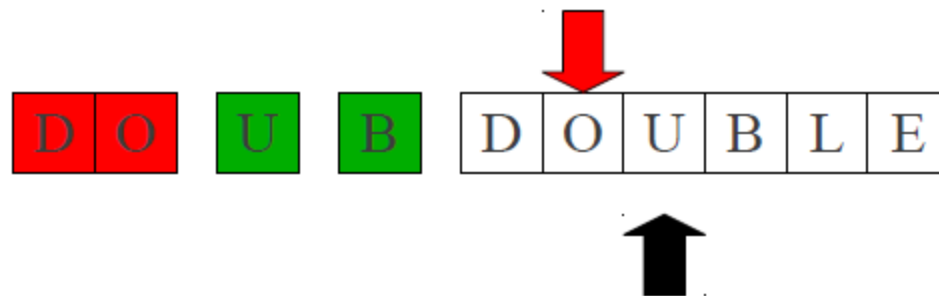
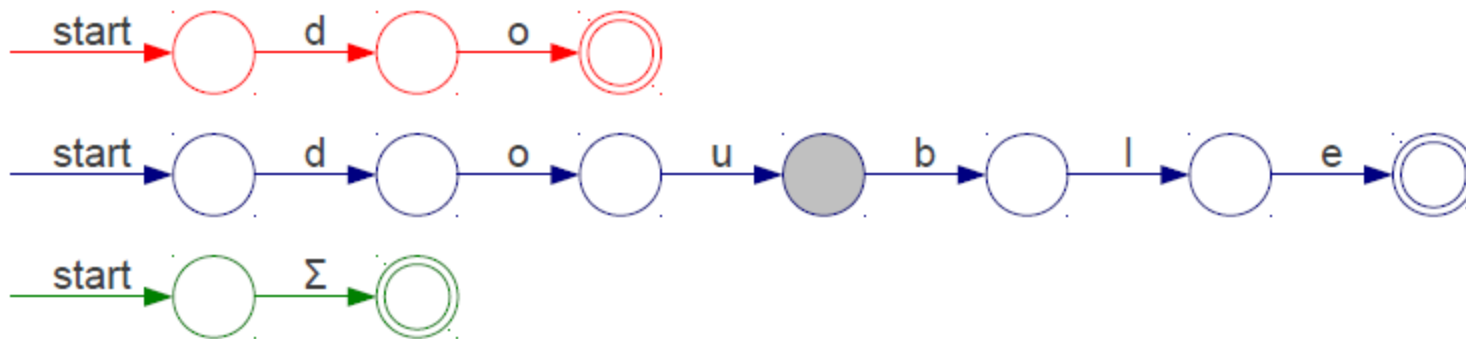
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

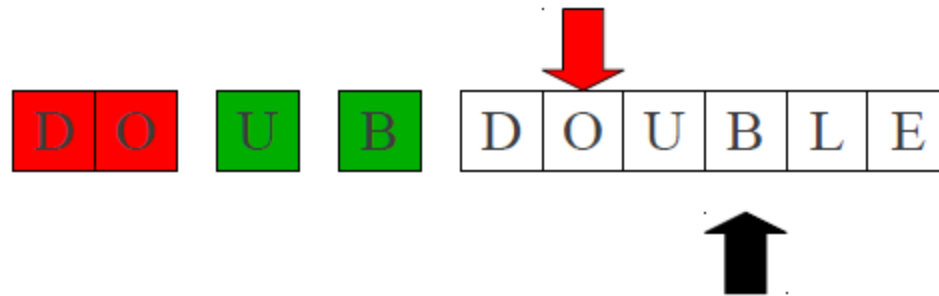
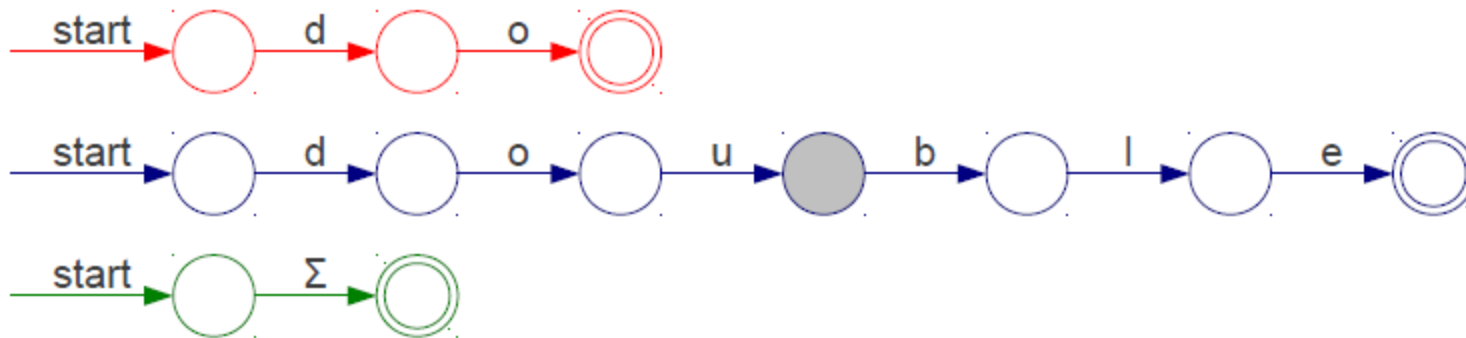
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

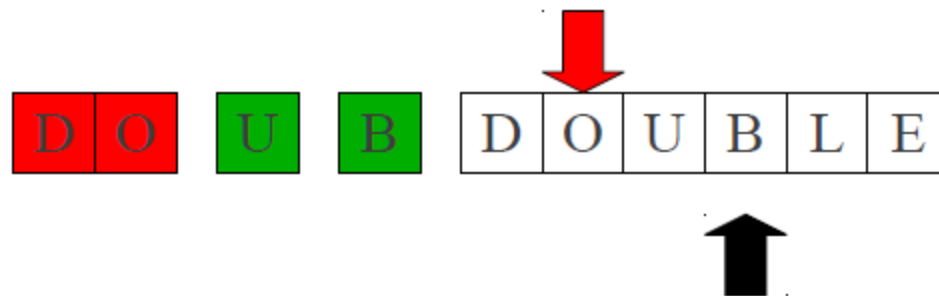
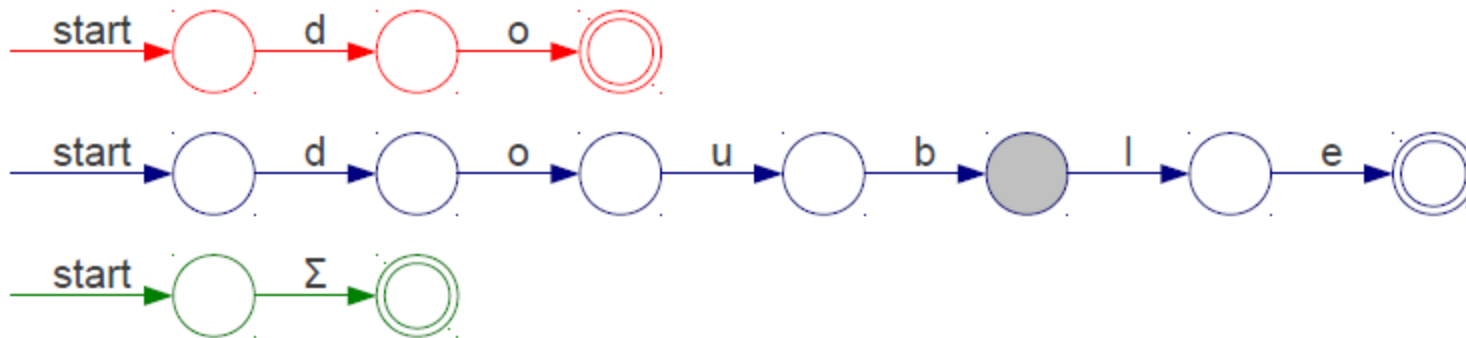
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

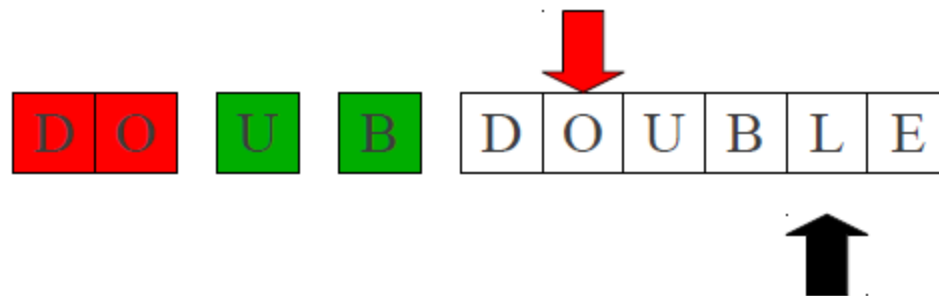
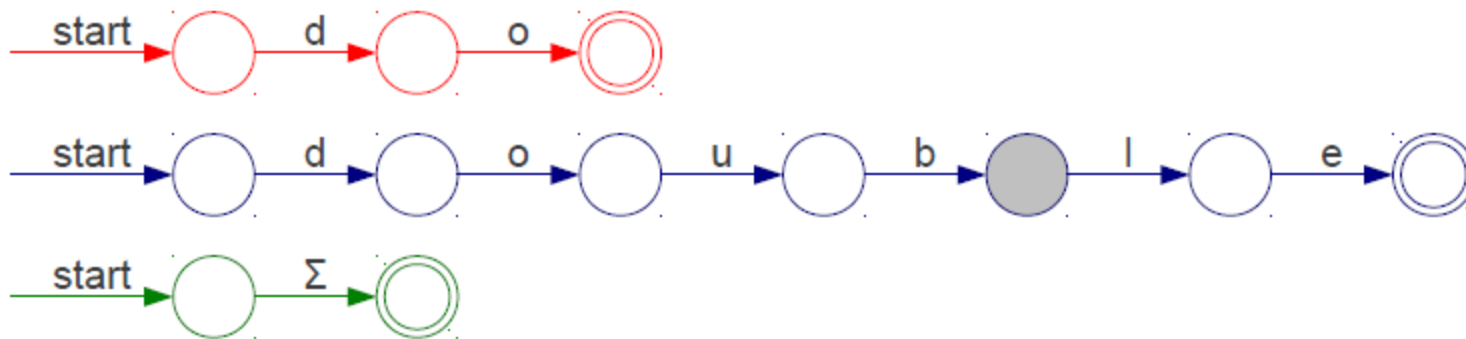
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

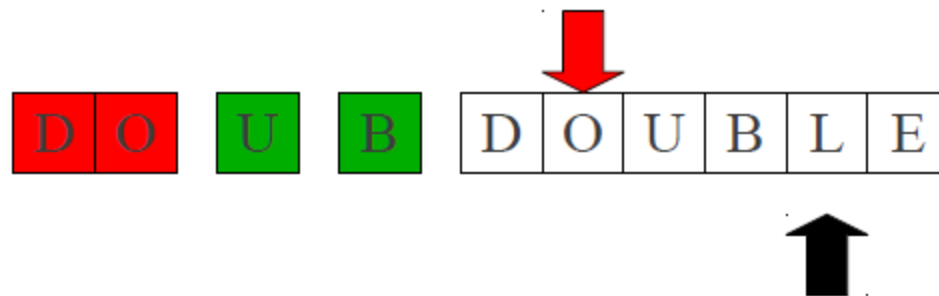
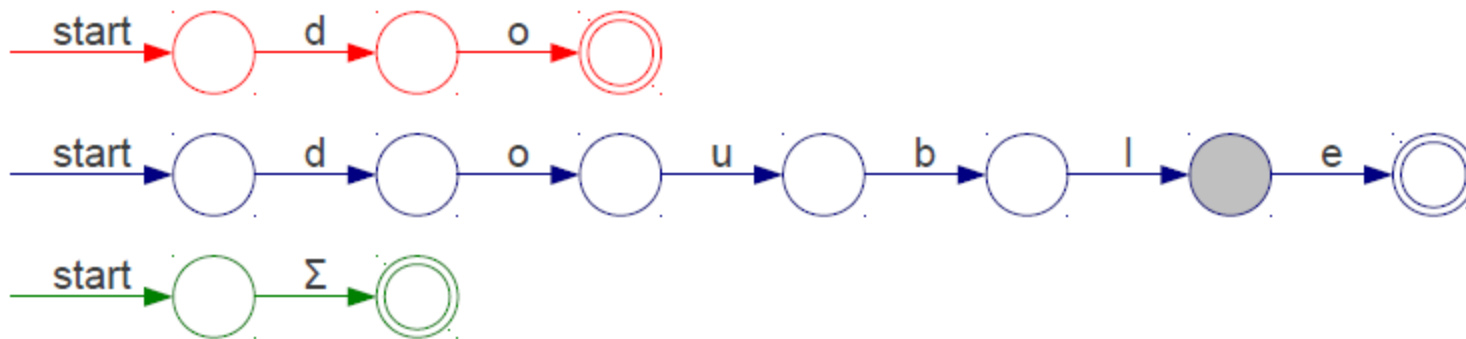
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

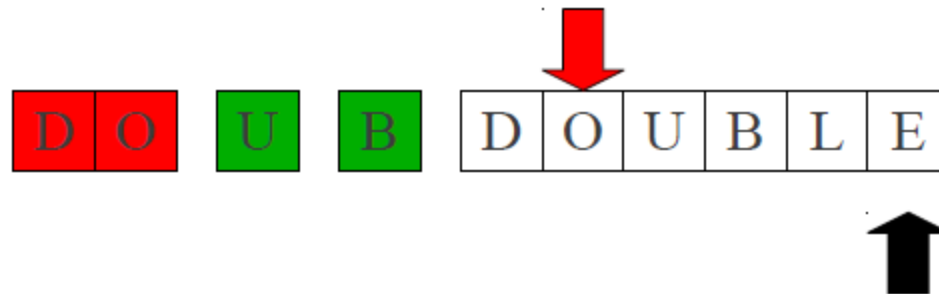
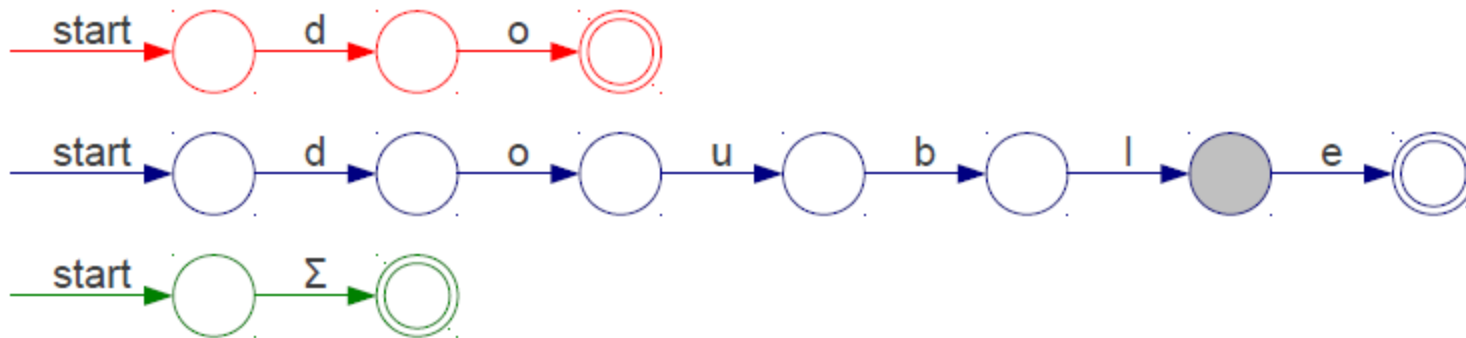
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

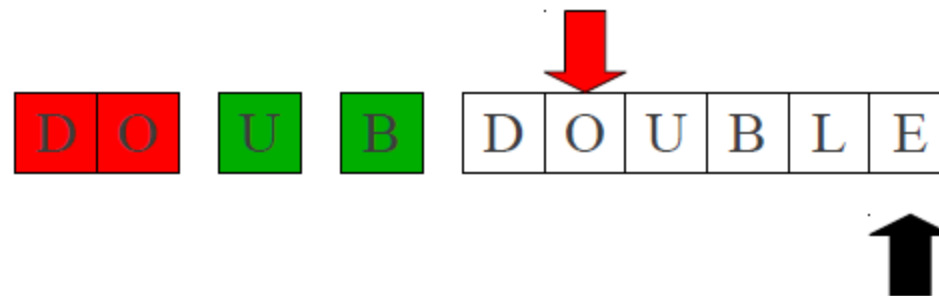
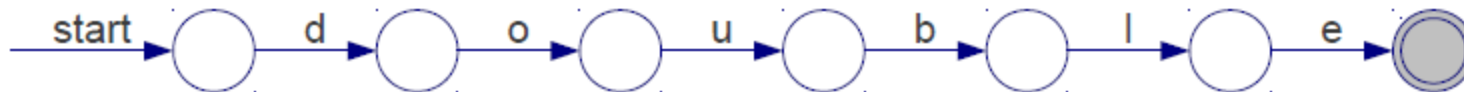
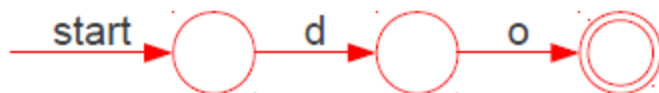
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

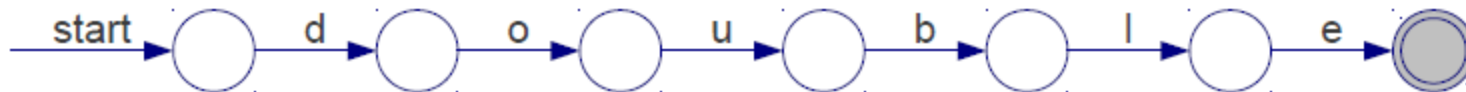
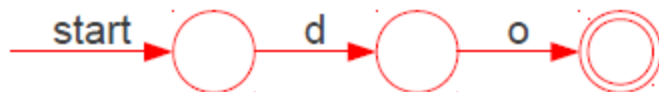
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

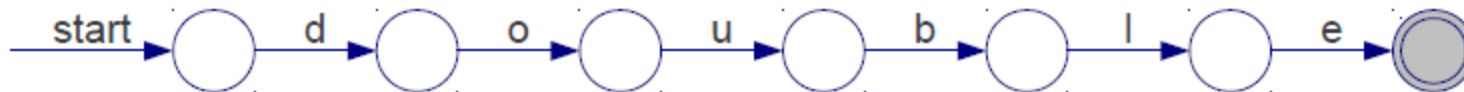
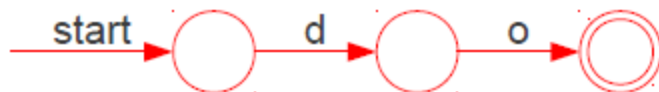
do

T_Double

double

T_Mystery

[A-Za-z]



Cài đặt giải thuật Maximal munch

T_Do

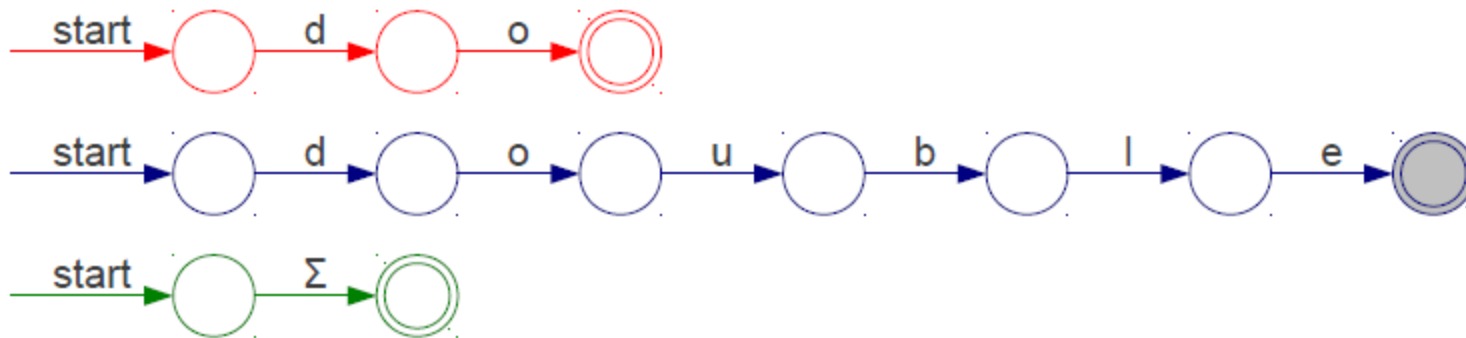
do

T_Double

double

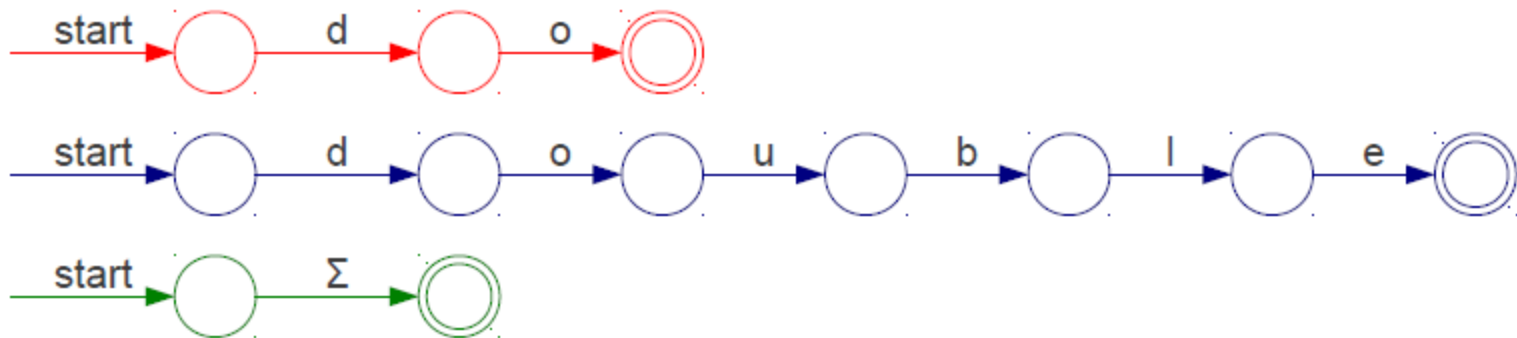
T_Mystery

[A-Za-z]

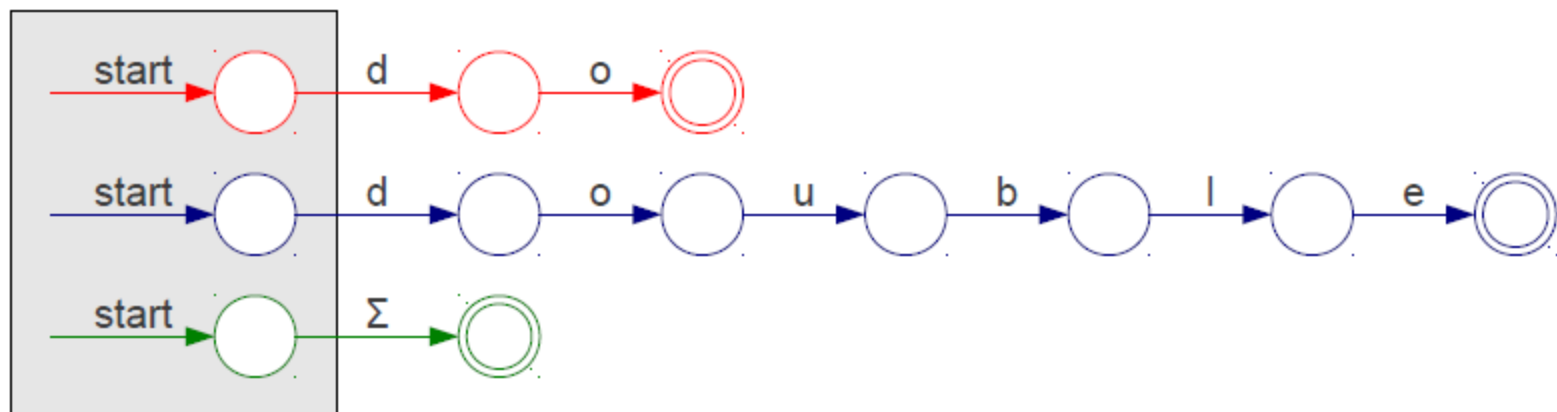


D O U B D O U B L E

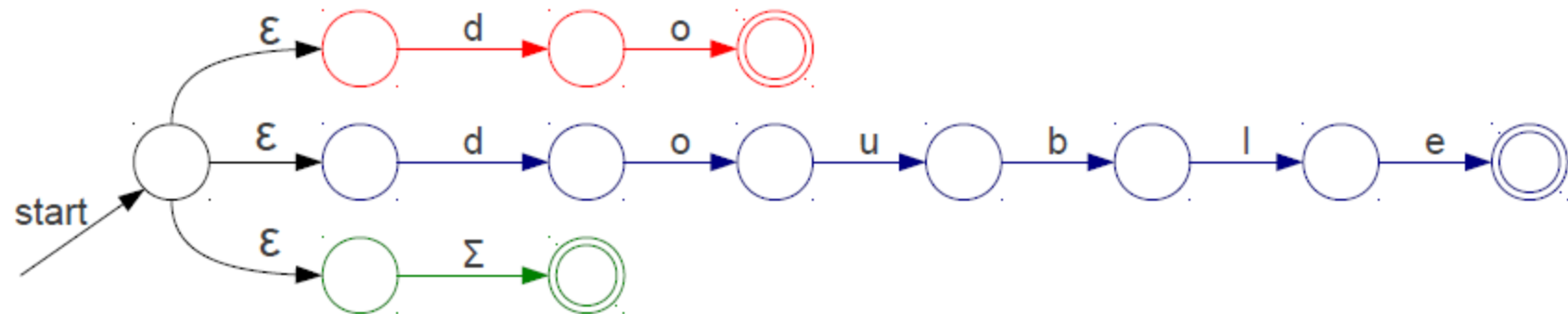
Cải tiến



Cải tiến

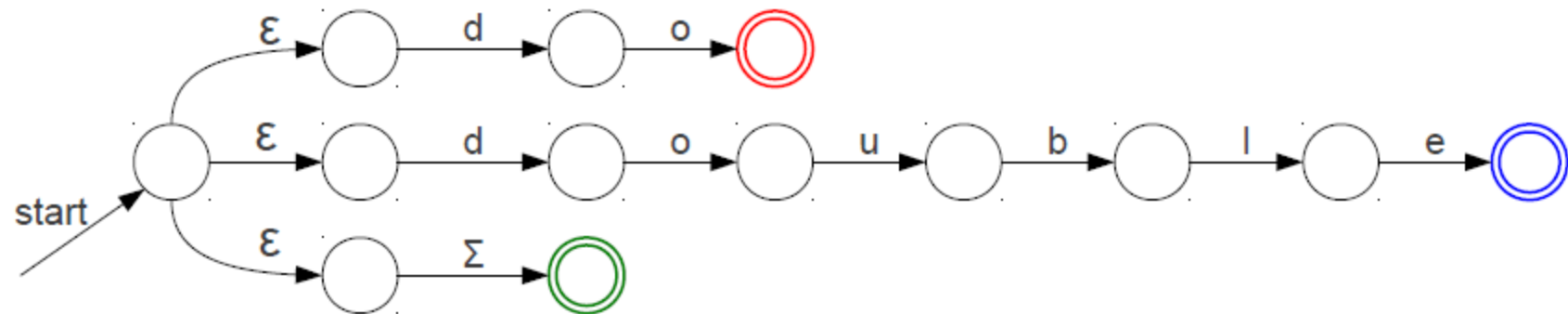


Cải tiến



Xây dựng 1 automat duy nhất.

Cải tiến



Gán nhãn cho mỗi trạng thái kết thúc tương ứng với các automata ban đầu.

Một mơ hồ khác nữa

T_Do

do

T_Double

double

T_Identifier [A-Za-z_] [A-Za-z0-9_]*

d	o	u	b	l	e
---	---	---	---	---	---

d	o	u	b	l	e
d	o	u	b	l	e

Trả về token nào ?

Một số luật tách khác

- Khi có 2 biểu thức chính quy có thể được áp dụng, chọn biểu thức có độ ưu tiên lớn
- Luật ưu tiên đơn giản: chọn luật được định nghĩa trước

Một số luật tách khác

T_Do do
T_Double double
T_Identifier [A-Za-z_][A-Za-z0-9_]*

d	o	u	b	l	e
---	---	---	---	---	---

d	o	u	b	l	e
d	o	u	b	l	e

Một số luật tách khác

T_Do do
T_Double double
T_Identifier [A-Za-z_][A-Za-z0-9_]*

d	o	u	b	l	e
---	---	---	---	---	---

d	o	u	b	l	e
---	---	---	---	---	---

Vấn đề sau cùng

- Ta đã biết cách giải quyết khi có nhiều luật khớp với đầu vào.
- Nếu không có luật nào khớp với đầu vào, ta sẽ làm gì ?
 - Mẹo: thêm một luật “catch-all” khớp với bất kỳ ký tự nào và xuất ra lỗi.

```
[0-9]+    printf("Integer number\n");  
.         printf("Unrecognized character: %s\n", yytext);
```


Tổng kết về giải quyết độ

- Xây dựng automat cho mỗi biểu thức chính quy.
- Kết hợp tất cả các automat thành một automat duy nhất bằng cách thêm vào một trạng thái bắt đầu mới.
- Duyệt đầu vào, giữ lại lần khớp sau cùng
- Tách đầu vào bằng cách chọn luật có độ ưu tiên cao nhất.
- Thêm một luật catch-all để xử lý trường hợp lỗi từ vựng.

Tổng kết chương

- Phân tích từ vựng tách đầu vào thành các **token**. Mỗi token sở hữu 1 **lexeme** và các **thuộc tính**.
- Lexemes là tập các chuỗi ký tự được định nghĩa bằng **biểu thức chính quy**.
- Biểu thức chính quy có thể được xây dựng thành các automat (đơn định hoặc không đơn định)
- Giải thuật Maximal munch sử dụng automat cho phép phân tích từ vựng nhanh.
- Không phải token nào cũng được lấy trực tiếp từ chương trình nguồn (ngôn ngữ python)